This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

Nir Shlezinger, Guy Revach, Anubhab Ghosh, Saikat Chatterjee, Shuo Tang, Tales Imbiriba, Jindrich Dunik, Ondrej Straka, Pau Closas, and Yonina C. Eldar

Artificial Intelligence-Aided Kalman Filters

AI-Augmented Designs for Kalman-Type Algorithms

xxxxx

The Kalman filter (KF) and its variants are among the most celebrated algorithms in signal processing. These methods are used for state estimation of dynamic systems by relying on mathematical representations in the form of simple state-space (SS) models, which may be crude and inaccurate descriptions of the underlying dynamics. Emerging data-centric artificial intelligence (AI) techniques tackle these tasks using deep neural networks (DNNs), which are model agnostic. Recent developments illustrate the possibility of fusing DNNs with classic Kalman-type filtering, obtaining systems that learn to track in partially known dynamics. This article provides a tutorial-style overview of design approaches for incorporating AI in aiding KF-type algorithms. We review both generic and dedicated DNN architectures suitable for state estimation and provide a systematic presentation of techniques for fusing AI tools with KFs and for leveraging partial SS modeling and data, categorizing design approaches into task oriented and SS model oriented. The usefulness of each approach in preserving the individual strengths of model-based KFs and data-driven DNNs is investigated in a qualitative and quantitative study (whose code is publicly available), illustrating the gains of hybrid model-based/data-driven designs. We also discuss existing challenges and future research directions that arise from fusing AI and Kalman-type algorithms.

C. Candan acted as the associate editor for this article and approved it for publication.

Digital Object Identifier 10.1109/MSP.2025.3569395

Introduction

The Apollo program, which successfully landed the first humans on the moon, is still considered one of mankind's greatest achievements. Among the technological innovations that played a role in the success of the Apollo program is a filtering method based on the extended version of an algorithm developed by Rudolf Kalman in the late 1950s [1], which was used to track and estimate the trajectory of the spaceship. This algorithm, known as the *KF*, and its extension into the extended KF (EKF) [2], provide an accurate and reliable real-time trajectory estimation while still being simple to implement and applicable on the hardware-limited navigation computer of the Apollo spaceship [3]. To date, the KF is among the most celebrated algorithms in signal processing and electrical engineering at large, with numerous applications, including radar, biomedical

1053-5888 © 2025 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies.

Authorized licensed use limited to: The Chinese University of Hong Kong CUHK(Shenzhen). Downloaded on September 15,2025 at 16:58:26 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

systems, vehicular technology, navigation, and wireless communications.

The KF is a model-based method; namely, it leverages mathematical parametric representations of the environment. Specifically, the KF and its variants [4] rely on the ability to describe the underlying dynamics as an SS model. Such model-based designs have several core advantages when the dynamics are faithfully captured using a known and tractable SS representation.

1) They can achieve optimal performance. For example, the KF achieves the minimum mean square error (MSE) for linear Gaussian SS models.
2) Model-based methods operate with controllable and often reduced complexity. In fact, the KF and its variants are implemented on devices such as sensors and mobile systems, where they operate in real time using limited computational and power resources.
3) Decisions made by the KF and its variants are interpretable in the sense that their internal features have concrete meaning and that their reasoning can be explained based on the observations and the SS model.
4) They are inherently adaptive, as changes in the SS model are naturally incorporated into the operation.
5) They reliably characterize uncertainty in their estimate, providing the error covariance alongside their estimates [5].

A key characteristic of KF-type algorithms is their reliance on knowledge of the underlying dynamics and, specifically, on the ability to accurately capture these dynamics using a known and tractable SS representation. A common approach is to rely on simplifying assumptions (e.g., linear systems, Gaussian mutually and temporally independent process and measurement noises, and so on) that make models understandable and the associated algorithms computationally efficient and then use data to estimate the unknown parameters. However, simple models frequently fail to represent some of the nuances and subtleties of dynamic systems and their associated signals. Practical applications are thus often required to operate with partially known SS models. Furthermore, the performance and reliability of KF-type algorithms degrade considerably when using a postulated SS model that deviates from the true nature of the underlying system.

The unprecedented success of machine learning (ML), and particularly deep learning, as the enabler technology for AI in areas such as computer vision has initiated a general mindset geared toward data. It is now quite common practice to replace simple principled models with data-driven pipelines, employing ML architectures based on DNNs that are trained with massive volumes of labeled data. DNNs can be trained end to end without relying on analytical approximations, and therefore, they can operate in scenarios where analytical models are unknown or highly complex [6]. With the growing popularity of deep learning, recent years have witnessed the design of various DNNs for tasks associated with KF-type algorithms, including, e.g., the application of DNNs for state estimation [7], [8] and control [9].

AI systems can learn from data to track dynamic systems without relying on full knowledge of underlying SS models. However, replacing KF-type algorithms with deep architectures gives rise to several core challenges.

1) The computational burden of training and utilizing highly parameterized DNNs as well as the fact that massive datasets are typically required for their training often constitute major drawbacks in various signal processing applications. This limitation is particularly relevant for hardware-constrained devices (such as mobile systems and sensors), whose ability to utilize highly parameterized DNNs is typically limited [10].
2) Due to the complex and generic structure of DNNs, it is often challenging to understand how they obtain their predictions or to track the rationale leading to their decisions.
3) DNNs typically struggle to adapt to variations in the data distribution and are limited in their capability to provide an estimation of uncertainty.

The challenges outlined above gave rise to a growing interest in using AI not to replace KF-type algorithms but to empower them as a form of hybrid model-based/data-driven design [11]. Various approaches have been proposed for combining KFs and deep learning, including training DNNs to map data into features that obey a known SS model and can be tracked with a KF [12], [13], employing deep models for identifying SS models to be used for KF-based tracking [14], [15], and converting the KF into an ML model that can be trained in a supervised [16], [17], [18], [19] or unsupervised [20], [21] manner. Such AI-empowered KFs are already in various signal processing applications, ranging from brain–machine interfaces, acoustic echo cancellation, and financial monitoring to beam tracking in wireless systems and drone-based monitoring systems [22], [23], [24]. These recent advances in combining AI and KFs, along with their implications on emerging technologies, motivate a systematic presentation of the different design approaches as well as the designs' associated signal processing challenges.

In this article, we provide a tutorial-style overview of the design approaches and potential benefits of combining deep learning tools with KF-type algorithms. We refer to this as "AI-aided KFs." While highlighting the potential benefits of AI-aided KFs, we also mention the main signal processing challenges that arise from such hybrid designs. For this goal, we begin by reviewing the fundamentals of the KF that are relevant to understanding its fusion with AI. We briefly describe its core statistical representation—the SS model—and formulate the mathematical steps for filtering and smoothing. We then discuss the pros and cons associated with KF-type algorithms and pinpoint the main challenging aspects that motivate the usage of AI tools. Next, we shift our focus to deep learning techniques that are suitable for processing time sequences, briefly reviewing both generic time sequence architectures, such as recurrent neural networks (RNNs) and attention mechanisms, and proceeding to DNN architectures that are inspired by SS representations [25] and by KF processing flow [7]. We

Authorized licensed use limited to: The Chinese University of Hong Kong CUHK(Shenzhen). Downloaded on September 15,2025 at 16:58:26 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

discuss the gains offered by such data-driven techniques while also highlighting their limitations, in turn indicating the potential of combining AI techniques with classic SS model-based KF-type methods.

The bulk of the article is dedicated to presenting design approaches that combine deep learning techniques with KF-type processing based on a partial mathematical representation of the dynamics. We categorize existing design approaches, drawing inspiration from the ML paradigms of discriminative (task-oriented) learning and generative (statistical model-oriented) learning [26]. Accordingly, the first part is dedicated to AI-augmented KFs via task-oriented learning, presenting in detail candidate approaches for converting KF-type filtering into an ML architecture via DNN augmentation. These include designs that employ an external DNN, either sequentially [27] or in parallel [19], as well as DNNs augmented into a KF-type algorithm, e.g., for Kalman gain (KG) computation [16]. We then proceed to detail SS model-oriented AI-aided KF designs. These can be viewed as a form of AI-aided system identification, i.e., techniques that utilize DNNs in the process of identifying SS models, which can then be used by model-based KF-type tracking [14], [15], [28]. We discuss several approaches with various structures connecting physics-based and data-based model components, such as incremental, hybrid, or fixed SS structures.

To highlight the gains of each approach for designing AI-aided KFs and capture the interplay between the different algorithms and conventional model-based or contemporary data-driven methods, we provide a comparative study. We begin with a qualitative comparison, pinpointing the conceptual differences among the design approaches in terms such as the level of domain knowledge required, the type of data needed, and flexibility. We also provide a quantitative comparison, where we evaluate representative methods from the presented design approaches in a setting involving the tracking of the challenging Lorenz attractor. The article concludes with a discussion of open challenges and future research directions.

Notations

We use boldface lowercase for vectors, e.g., $\boldsymbol{x}$, and boldface uppercase letters, e.g., $\boldsymbol{X}$, for matrices. For a time sequence $\boldsymbol{x}_t$ and time indices $t_1 \leq t_2$, we use the abbreviated form $\boldsymbol{x}_{t_1:t_2}$ for the set of variables $\{\boldsymbol{x}_t\}_{t=t_1}^{t_2}$. We use $\mathcal{N}(\boldsymbol{\mu},\boldsymbol{\Sigma})$ for the multivariate Gaussian distribution with mean $\boldsymbol{\mu}$ and covariance $\boldsymbol{\Sigma}$, while $p(\cdot)$ is a probability density function, and $\|\boldsymbol{x}\|_{\boldsymbol{C}}^2$ is the squared ℓ_2 norm of $\boldsymbol{x}$ weighted by the matrix $\boldsymbol{C}$; i.e., $\|\boldsymbol{x}\|_{\boldsymbol{C}}^2 = \boldsymbol{x}^\top \boldsymbol{C}\boldsymbol{x}$. The operations $(\cdot)^\top$ and $\|\cdot\|_2$ are the transpose and ℓ_2 norm, respectively.

Fundamentals of Kalman filtering

In this section, we review some basics of Kalman-type filtering. We commence with reviewing SS models, after which we recall the formulation of the KF and EKF. We conclude this section by highlighting the challenges that motivate Kalman-type filtering's augmentation with deep learning.

SS models

SS models are a class of mathematical models that describe the probabilistic behavior of dynamical systems. They constitute the fundamental framework for formulating a broad range of engineering problems in the areas of signal processing, control, and communications, and they are also widely used in environment studies, economics, and many more. The core of SS models lies in the representation of the dynamics of a system using a latent state variable that evolves over time while being related to the observations made by the system.

Generic SS models

Focusing on discrete-time formulations of continuous-valued variables, SS models represent the interplay among the observations at time t, denoted $\boldsymbol{y}_t$; an input signal $\boldsymbol{u}_t$; and a state capturing the system dynamics $\boldsymbol{x}_t$. In general, SS models consist of 1) a state evolution model of the form

$$\boldsymbol{x}_{t+1} = \tilde{f}_t(\boldsymbol{x}_t, \boldsymbol{u}_t, \boldsymbol{v}_t) \tag{1a}$$

representing how the state evolves in time, with $\boldsymbol{v}_t$ being the process noise, and 2) an observation model

$$\boldsymbol{y}_t = \tilde{h}_t(\boldsymbol{x}_t, \boldsymbol{w}_t) \tag{1b}$$

which relates the observations and the current system state, where $\boldsymbol{w}_t$ represents the measurement noise. While the mappings $\tilde{f}_t(\cdot)$ and $\tilde{f}_t(\cdot)$ are deterministic, stochasticity is induced by the temporally independent noises $\boldsymbol{v}_t$ and $\boldsymbol{w}_t$ and by the distribution of the initial state $\boldsymbol{x}_0$.

Gaussian SS models

A common special case of the above generic model is that referred to as the *linear Gaussian SS model*, used by the celebrated KF. In this special case of (1), the state evolution takes the form

$$\boldsymbol{x}_{t+1} = \boldsymbol{F}_t\boldsymbol{x}_t + \boldsymbol{G}_t\boldsymbol{u}_t + \boldsymbol{v}_t \tag{2a}$$

and the observation model is given by

$$\boldsymbol{y}_t = \boldsymbol{H}_t\boldsymbol{x}_t + \boldsymbol{w}_t \tag{2b}$$

where $\boldsymbol{F}_t, \boldsymbol{G}_t, \boldsymbol{H}_t$ are matrices of appropriate dimensions and $\boldsymbol{v}_t$ and $\boldsymbol{w}_t$ are temporally and mutually independent zero-mean Gaussian signals, with covariances $\boldsymbol{Q}_t$ and $\boldsymbol{R}_t$, respectively. Namely, $\boldsymbol{v}_t \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{Q}_t)$ and $\boldsymbol{w}_t \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{R}_t)$, while $\boldsymbol{v}_t$ is independent of $\boldsymbol{w}_t$, and both are independent of $\boldsymbol{v}_\tau$ and $\boldsymbol{w}_\tau$ for any $\tau \neq t$. The initial state is independent of the noises and assumed to obey $\boldsymbol{x}_0 \sim \mathcal{N}(\hat{\boldsymbol{x}}_0, \hat{\boldsymbol{\Sigma}}_0)$, with known $\hat{\boldsymbol{x}}_0, \hat{\boldsymbol{\Sigma}}_0$.

Similar models to (2) are frequently utilized for settings characterized by nonlinear transformations. In nonlinear SS

Authorized licensed use limited to: The Chinese University of Hong Kong CUHK(Shenzhen). Downloaded on September 15,2025 at 16:58:26 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

models with additive Gaussian noises (termed henceforth as *nonlinear Gaussian*), (1) takes the form

$$\boldsymbol{x}_{t+1} = f_t(\boldsymbol{x}_t, \boldsymbol{u}_t) + \boldsymbol{v}_t \tag{3a}$$

$$\boldsymbol{y}_t = h_t(\boldsymbol{x}_t) + \boldsymbol{w}_t \tag{3b}$$

where the noise signals are as in (2), i.e., temporally independent, with $\boldsymbol{v}_t \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{Q}_t)$ and $\boldsymbol{w}_t \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{R}_t)$.

Tasks

SS models are mostly associated with two main families of tasks. The first is state estimation, which deals with the recovery of the state variable $\boldsymbol{x}_t$ based on a set of observations $\{\boldsymbol{y}_\tau\}$, i.e., an open-loop system where the input $\boldsymbol{u}_t$ is either absent or not controlled. Common state estimation tasks include the following [4]:

1) *Filtering*: Estimate $\boldsymbol{x}_t$ from $\boldsymbol{y}_{1:t}$.
2) *Smoothing:* Estimate $\boldsymbol{x}_{1:T}$ from $\boldsymbol{y}_{1:T}$ for some $T > 0$.

Additional related tasks are prediction, input recovery, and imputation. State estimation plays a key role in applications that involve tracking, localization, and denoising.

The second family of SS model-based tasks includes those that deal with stochastic control (closed-loop) policies. In this family, the SS framework is used to select how to set the input variable $\boldsymbol{u}_t$ based on, e.g., past measurements $\boldsymbol{y}_{1:t}$. Such tasks are fundamental in robotics, vehicular systems, and aerospace engineering. As stochastic control policies often employ state estimation schemes followed by state regulators, we focus in this article on state estimation (i.e., the first family of tasks).

Kalman filtering

The representation of dynamic systems via SS models gives rise to some of the most celebrated algorithms in signal processing, particularly the family of Kalman-type filters. To describe these, we henceforth focus on state estimation. Therefore, for convenience, we omit the input signal $\boldsymbol{u}_t$ from the following relations (as it can also be absorbed into the process noise as a known bias term).

KF

The KF is the minimum MSE estimator for the filtering task in linear Gaussian SS models, i.e., estimating $\boldsymbol{x}_t$ from $\boldsymbol{y}_{1:t}$ when these are related via (2). In time step t, the KF estimates $\boldsymbol{x}_t$ using the previous estimate $\hat{\boldsymbol{x}}_{t-1}$ as a sufficient statistic and the observed $\boldsymbol{y}_t$; thus, its complexity does not grow in time.

The KF updates its estimates of the first- and second-order statistical moments of the state, which at time t are denoted by $\hat{\boldsymbol{x}}_{t|t}$ and $\boldsymbol{\Sigma}_{t|t}$, respectively. For $t = 0$, these moments are initialized to those of the initial state, namely, $\hat{\boldsymbol{x}}_0$ and $\hat{\boldsymbol{\Sigma}}_0$, while for $t > 0$, the estimates are obtained via a two-step procedure.

1) *Prediction*: The first step predicts the first- and second-order statistical moments of the current a priori state and observation, based on the previous a posteriori estimate. Specifically, at time t, the predicted moments of the state are computed via

$$\hat{\boldsymbol{x}}_{t|t-1} = \boldsymbol{F}_{t-1} \cdot \hat{\boldsymbol{x}}_{t-1|t-1} \tag{4a}$$

$$\boldsymbol{\Sigma}_{t|t-1} = \boldsymbol{F}_{t-1} \cdot \boldsymbol{\Sigma}_{t-1|t-1} \cdot \boldsymbol{F}_{t-1}^{\mathsf{T}} + \boldsymbol{Q}_{t-1}. \tag{4b}$$

The predicted moments of the observations are computed as

$$\hat{\boldsymbol{y}}_{t|t-1} = \boldsymbol{H}_t \cdot \hat{\boldsymbol{x}}_{t|t-1} \tag{5a}$$

$$\boldsymbol{S}_{t|t-1} = \boldsymbol{H}_t \cdot \boldsymbol{\Sigma}_{t|t-1} \cdot \boldsymbol{H}_t^{\mathsf{T}} + \boldsymbol{R}_t. \tag{5b}$$

2) *Update*: The predicted a priori moments are updated using the current observation $\boldsymbol{y}_t$ in the a posteriori state moments. The updated moments are computed as

$$\hat{\boldsymbol{x}}_{t|t} = \hat{\boldsymbol{x}}_{t|t-1} + \boldsymbol{K}_t \cdot \triangle \boldsymbol{y}_t \tag{6a}$$

$$\boldsymbol{\Sigma}_{t|t} = \boldsymbol{\Sigma}_{t|t-1} - \boldsymbol{K}_t \cdot \boldsymbol{S}_{t|t-1} \cdot \boldsymbol{K}_t^{\mathsf{T}}. \tag{6b}$$

Here, $\boldsymbol{K}_t$ is the KG, and it is given by

$$\boldsymbol{K}_t = \boldsymbol{\Sigma}_{t|t-1} \cdot \boldsymbol{H}_t^{\mathsf{T}} \cdot \boldsymbol{S}_{t|t-1}^{-1}. \tag{7}$$

The term $\triangle \boldsymbol{y}_t \triangleq \boldsymbol{y}_t - \widehat{\boldsymbol{y}}_{t|t-1}$ is then defined as the innovation, representing the difference between the predicted observation and the observed value.

The output of the KF is the estimated state, i.e., $\hat{\boldsymbol{x}}_t \equiv \hat{\boldsymbol{x}}_{t|t}$, and the error covariance is $\boldsymbol{\Sigma}_t = \boldsymbol{\Sigma}_{t|t}$. Various proofs are provided in the literature for the MSE optimality of the KF [4]. A relatively compact way to prove this follows from the more general Bayes' rule and Chapman–Kolmogorov equation, recalled in "From Chapman–Kolmogorov to the Kalman Filter."

Extension to smoothing

While the KF is formulated for the filtering task, it also naturally extends (while preserving its optimality) to smoothing tasks. A leading approach to realize a smoother is the Rauch–Tung–Striebel (RTS) algorithm [5], which employs two subsequent recursive passes to estimate the state, termed the *forward pass* and *backward pass*. The forward pass is the standard KF, while the backward pass refines the estimates for each time t using future observations at time instants $\tau \in \{t+1, \ldots, T\}$.

The backward pass is similar in its structure to the update step in the KF. For the time step $t = T$, it uses $\hat{\boldsymbol{x}}_{T|T}$ and $\boldsymbol{\Sigma}_{T|T}$, computed by the forward recursion of the KF. Then, it carries out a backward recursion, where for each $t \in \{T-1, \ldots, 1\}$, the forward belief is corrected with future estimates via

$$\hat{\boldsymbol{x}}_{t|T} = \hat{\boldsymbol{x}}_{t|t} + \overleftarrow{\boldsymbol{K}}_t \cdot (\hat{\boldsymbol{x}}_{t+1|T} - \hat{\boldsymbol{x}}_{t+1|t}) \tag{8a}$$

$$\boldsymbol{\Sigma}_{t|T} = \boldsymbol{\Sigma}_{t|t} - \overleftarrow{\boldsymbol{K}}_t \cdot (\boldsymbol{\Sigma}_{t+1|T} - \boldsymbol{\Sigma}_{t+1|T}) \cdot \overleftarrow{\boldsymbol{K}}_t^{\mathsf{T}}. \tag{8b}$$

Here, $\overleftarrow{\boldsymbol{K}}_t$ is the backward KG, computed based on second-order statistical moments from the forward pass as

$$\overleftarrow{\boldsymbol{K}}_t = \boldsymbol{\Sigma}_{t|t} \cdot \boldsymbol{F}_t^{\mathsf{T}} \cdot \boldsymbol{\Sigma}_{t+1|t}^{-1}. \tag{9}$$

Authorized licensed use limited to: The Chinese University of Hong Kong CUHK(Shenzhen). Downloaded on September 15,2025 at 16:58:26 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

From Chapman–Kolmogorov to the Kalman Filter

Consider a generic state-space (SS) model as in (1) without an input signal $\boldsymbol{u}_t$. By Bayes' rule and the Markovian nature of (1), it holds that the conditional distribution of $\boldsymbol{u}_t$ satisfies [S1]

$$p(\boldsymbol{x}_t|\boldsymbol{y}_{1:t}) = \frac{p(\boldsymbol{y}_t|\boldsymbol{x}_t)p(\boldsymbol{x}_t|\boldsymbol{y}_{1:t-1})}{p(\boldsymbol{y}_t|\boldsymbol{y}_{1:t-1})} \quad \text{(S1a)}$$

where

$$p(\boldsymbol{y}_t|\boldsymbol{y}_{1:t-1}) = \int p(\boldsymbol{y}_t|\boldsymbol{x}_t)p(\boldsymbol{x}_t|\boldsymbol{y}_{1:t-1})d\boldsymbol{x}_t \quad \text{(S1b)}$$

$$p(\boldsymbol{x}_t|\boldsymbol{y}_{1:t-1}) = \int p(\boldsymbol{x}_t|\boldsymbol{x}_{t-1})p(\boldsymbol{x}_{t-1}|\boldsymbol{y}_{1:t-1})d\boldsymbol{x}_{t-1}. \quad \text{(S1c)}$$

Combining (S1a) and (S1b) with (S1c), referred to as the *Chapman–Kolmogorov equation* for SS models [5, Ch. 4.2], describes how the posterior $p(\boldsymbol{x}_{t-1}|\boldsymbol{y}_{1:t-1})$ is recursively updated into $p(\boldsymbol{x}_t|\boldsymbol{y}_{1:t})$.

For the special case of a linear Gaussian SS model as in (1), all the considered variables are jointly Gaussian. Specifically, if $\boldsymbol{x}_{t-1}|\boldsymbol{y}_{1:t-1} \sim \mathcal{N}(\hat{\boldsymbol{x}}_{t-1|t-1}, \Sigma_{t-1|t-1})$, then (S1c) implies that $\boldsymbol{x}_t|\boldsymbol{y}_{1:t-1} \sim \mathcal{N}(\hat{\boldsymbol{x}}_{t|t-1}, \Sigma_{t|t-1})$ [computed via (4)], which together with (S1b) indicates that $\boldsymbol{y}_t|\boldsymbol{y}_{1:t-1} \sim \mathcal{N}(\hat{\boldsymbol{y}}_{t|t-1}, \boldsymbol{S}_{t|t-1})$ [computed via (5)]. Combining these with (S1a) reveals that $\boldsymbol{x}_t|\boldsymbol{y}_{1:t} \sim \mathcal{N}(\hat{\boldsymbol{x}}_{t|t}, \Sigma_{t|t})$, with moments computed via (6). Accordingly, $\hat{\boldsymbol{x}}_t$ computed by the Kalman filter is exactly the conditioned expectation of $\boldsymbol{x}_t$ conditioned on $\boldsymbol{y}_{1:t}$; i.e., it is mean square error optimal.

Reference

[S1] D. Barber and A. T. Cemgil, "Graphical models for time-series," *IEEE Signal Process. Mag.*, vol. 27, no. 6, pp. 18–28, Nov. 2010, doi: 10.1109/MSP.2010.938028.

The output of the RTS is the estimated state for every time instant $t \in \{1, \ldots, T\}$, with $\hat{\boldsymbol{x}}_t \equiv \hat{\boldsymbol{x}}_{t|T}$, as well as the error covariance $\boldsymbol{\Sigma}_t \equiv \boldsymbol{\Sigma}_{t|T}$.

Extensions to nonlinear models

The KF is MSE optimal for linear and Gaussian SS models. For nonlinear settings, approaches vary between nonlinear Gaussian models, as in (3), and non-Gaussian settings.

Filtering algorithms for state estimation in nonlinear Gaussian SS models are typically designed to preserve the linear operation of the KF update step with respect to measurement. The key challenge in approximating the operation of the KF lies in propagation of the first- and second-order moments. Arguably the most common nonlinear variant of the KF, known as the *EKF*, is based on local linearizations. In the EKF, prediction of the first-order moments in (4a) and (5a) is replaced with

$$\hat{\boldsymbol{x}}_{t|t-1} = f_t(\hat{\boldsymbol{x}}_{t-1|t-1}), \quad \hat{\boldsymbol{y}}_{t|t-1} = \boldsymbol{h}_t(\hat{\boldsymbol{x}}_{t|t-1}). \quad (10)$$

For the second-order moments, the matrices $\boldsymbol{F}_t$ and $\boldsymbol{H}_t$ in the KF formulations are respectively replaced with the Jacobian matrices of $f_t(\cdot)$ and $h_t(\cdot)$; i.e.,

$$\hat{\boldsymbol{F}}_t = \nabla_{x_{t-1}} f_{t-1}(\boldsymbol{x}_{t-1})\,|_{x_{t-1}=\hat{x}_{t-1|t-1}} \quad (11a)$$

$$\hat{\boldsymbol{H}}_t = \nabla_{x_t} h_t(\boldsymbol{x}_t)\Big|_{x_t=\hat{x}_{t|t-1}}. \quad (11b)$$

Alternatively, the propagation of first- and second-order moments can be approximated using the unscented transform, resulting in the unscented KF (UKF), or using the cubature and Gauss–Hermite deterministic or stochastic quadrature rules.

The EKF and UKF are approximations of the KF designed for nonlinear SS models with Gaussian distributed noise. When the SS representation has non-Gaussian noise, state estimation algorithms typically aim at recursively updating the posterior distribution via the Chapman–Kolmogorov relation, without resorting to its Gaussian special case utilized by the KF (see "From Chapman–Kolmogorov to the Kalman Filter"). Leading algorithms that operate in this manner include the family of particle filters (PFs), which are based on sequential sampling; Gaussian sum filters, based on Gaussian sum representation of all densities; and point–mass filters, numerically solving the Bayesian relation in a typically rectangular grid.

Pros and cons of model-based KF-type algorithms

The KF and its variants are a family of widely utilized algorithms [5]. The core of these model-based methods is the SS model, i.e., the representation of the system dynamics via closed-form equations, such as those in (1). When the SS model faithfully captures the dynamics, these algorithms have several key desirable properties.

1) When the SS model is well described as being linear with Gaussian noise, KF-based algorithms can approach optimal state estimation performance in the sense of minimizing the MSE.
2) Model-based methods are inherently adaptive to known variations in the SS model. For instance, $\boldsymbol{H}_t$ can change with t, and one needs only to substitute the updated matrix in the corresponding equations.
3) Their operation is fully interpretable in the sense that one can associate the internal features with concrete interpretation, as they represent, e.g., statistical moments of prior and posterior predictions.
4) They provide reliable uncertainty measures via the error covariance in (6b) and (8b).
5) KF-type algorithms operate with relatively low complexity that does not grow with time.

Authorized licensed use limited to: The Chinese University of Hong Kong CUHK(Shenzhen). Downloaded on September 15,2025 at 16:58:26 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

However, the reliance of KF-type algorithms on faithful mathematical modeling of the underlying dynamics, and the algorithms' natural suitability with simplistic linear Gaussian models, also gives rise to several core challenges encountered in various applications. These challenges can be roughly categorized as follows:

1) The state evolution and observation models employed in SS models are often approximations of the true dynamics, whose fidelity may vary considerably among applications. For instance, while the temporal evolution of a state corresponding to the position and velocity of a vehicle can be represented as a linear transformation via mechanical relationships, e.g., a constant velocity model [15], such modeling of $f_t(\cdot)$ is inherently a crude first-order approximation. Similarly, the relationship between the velocity of a vehicle and its sensed motor currents can be captured via an observation model of the form (3); the exact specification of the mapping $h_t(\cdot)$ is likely to be elusive.
2) The purpose of the noise signals $\boldsymbol{v}_t$ and $\boldsymbol{w}_t$ is to 1) capture the inherent stochasticity in the state evolution and measurements, respectively, and 2) model the discrepancy between the SS representation and the true system. Their actual distribution is thus often unknown, complex, and possibly intractable. Non-Gaussianity can have a notable effect on performance and reliability, especially since Kalman-type algorithms seek a linear filtering operation.
3) Even when the dynamics are faithfully characterized by a non-linear Gaussian SS model, Kalman-type algorithms are suboptimal, with gaps from optimality largely depending on the nonlinearity.
4) Despite their relatively low complexity, nonlinear variants of the KF, such as the EKF and UKF, induce some latency during filtering. This is due to the need to carry out, e.g., local linearization and matrix inversion on each time instant (which the linear KF can do offline based on knowledge of the statistics).

The above characterization of the desirable properties and the challenges of model-based KF-type algorithms is summarized in Table 1. Specifically, the challenges characterized in C1–C4 motivate exploring data-driven approaches for tackling tasks associated with SS models, as detailed in the following section.

Combining AI with KFs

Recent years have witnessed the remarkable success of deep learning, being the main enabler for AI, in various applications involving processing of time sequences. Data-driven DNNs were shown to be able to catch the subtleties of complex processes and replace the need to explicitly characterize the domain of interest. Therefore, an alternative strategy to implement state estimation while coping with C1–C4, namely, without requiring explicit and accurate knowledge of the SS representation, is to learn this task from data using deep learning.

Time sequence filtering with DNNs

A common ML strategy utilizes highly parameterized abstract models trained from data to find the parameterization that minimizes the empirical risk (with regularization introduced to prevent overfitting). Their mapping, denoted $\mathcal{F}_\theta$, is dictated by a set of parameters denoted $\boldsymbol{\theta}$. In deep learning, the parametric model $\mathcal{F}_\theta$ is a DNN, with $\boldsymbol{\theta}$ being the network parameters. Such highly parameterized abstract models can effectively approximate any Borel measurable mapping, as follows from the universal approximation theorem [6, Ch. 6.4.1].

DNNs can learn various tasks from data without requiring mathematical representation of the underlying dynamics and observations model. Accordingly, DNNs applied for tasks such as filtering and smoothing do not require the formulation of the dynamics as an SS representation. Despite this invariance, the resulting architecture of the DNN can still draw some inspiration from SS representation and KF-type processing. Consequently, we divide our presentation into conventional DNNs, which account only for the fact that the data being processed are a time sequence, and SS/KF-inspired DNNs, with both families being invariant of the underlying model.

Conventional DNNs

Tasks associated with SS models, particularly filtering and smoothing, involve processing time sequences with temporal correlations. Accordingly, DNNs designed for time sequences can conceptually be trained to carry out these tasks. These architectures, discussed next, somewhat deviate from the basic DNN form, i.e., fully connected (FC) layers, which process fixed-size vectors rather than time sequences.

RNNs

A common DNN architecture for time sequences is based on RNNs. RNNs maintain an internal state vector, denoted $\boldsymbol{\theta}$, representing the system memory [6, Ch. 10]. The parametric mapping is given by

$$\hat{\boldsymbol{x}}_t = \mathcal{F}_\theta(\boldsymbol{y}_t, \boldsymbol{s}_{t-1}) \tag{12}$$

where the internal state also evolves via a learned parametric mapping

$$\boldsymbol{s}_t = \mathcal{G}_\theta(\boldsymbol{y}_t, \boldsymbol{s}_{t-1}). \tag{13}$$

Table 1. A summary of desired properties and challenges of model-based KF-type algorithms.

Desirable Properties		Challenges	
P1	Optimal for linear Gaussian models	C1	SS model is typically approximated
P2	Adaptable to known variations	C2	Noise model is often elusive
P3	Interpretable operation	C3	Suboptimal for nonlinear models
P4	Provides uncertainty	C4	Latency for nonlinear models
P5	Relatively low complexity		

Authorized licensed use limited to: The Chinese University of Hong Kong CUHK(Shenzhen). Downloaded on September 15,2025 at 16:58:26 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

The vanilla implementation of an RNN uses a hidden layer to map $\boldsymbol{y}_t$ and the latent $\boldsymbol{s}_{t-1}$ into an updated hidden variable $\boldsymbol{s}_t$. Alternative RNN architectures, such as gated recurrent units (GRUs) and long short-term memory (LSTM), employ several learned cells to update $\boldsymbol{s}_t$. Then, $\boldsymbol{s}_t$ is used to generate the instantaneous output $\hat{\boldsymbol{x}}_t$ using another layer, as illustrated in Figure 1(a).

Attention

A widely popular DNN architecture, which is the core of the transformer model, is based on attention mechanisms [30]. Such architectures were shown to be extremely successful in learning complex tasks in natural language processing, where the considered signals can be viewed as time sequences.

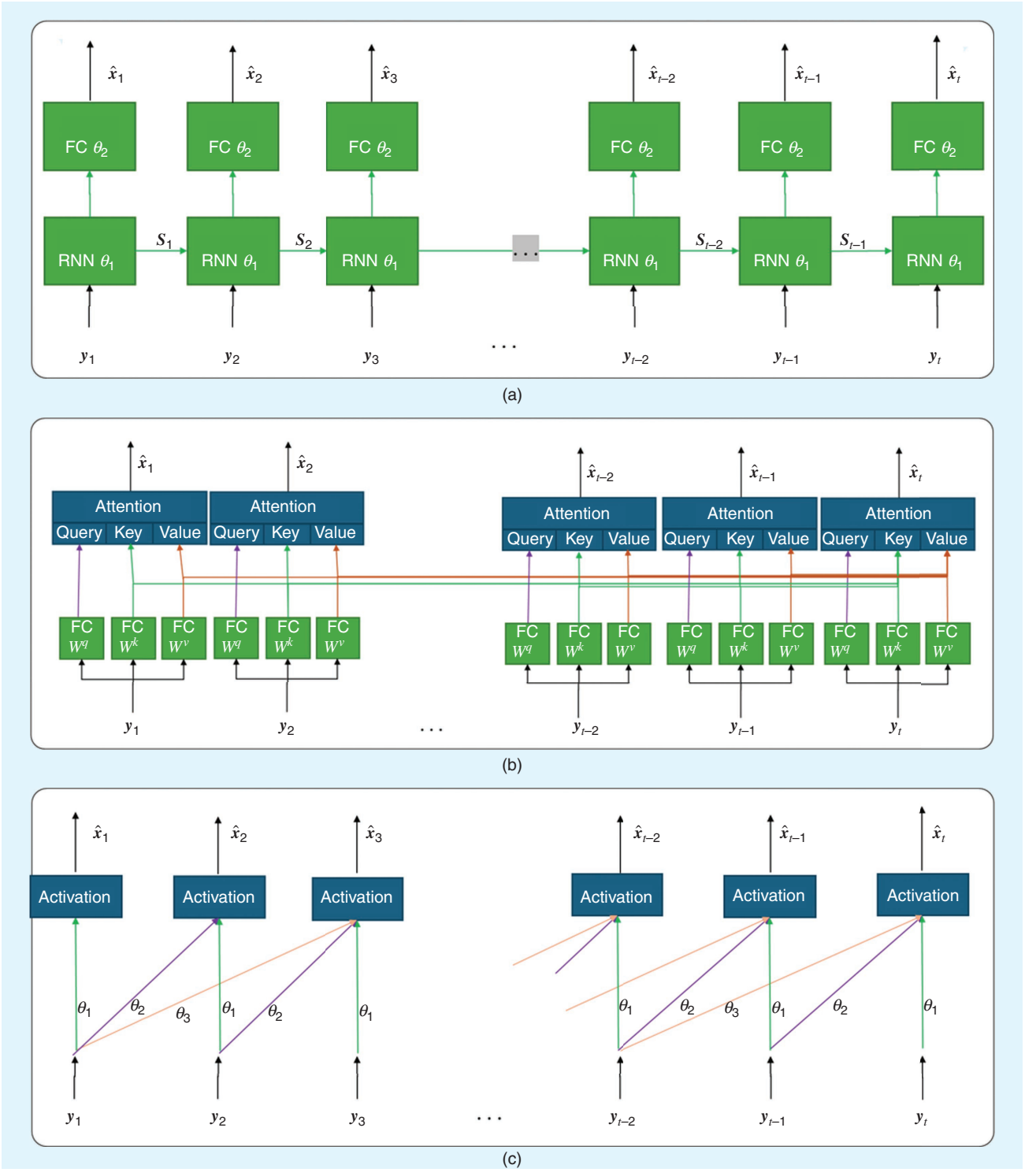

FIGURE 1. Conventional DNN architectures for tasks related to filtering and smoothing, including (a) RNNs, (b) (self-)attention, and (c) (1D) convolutional neural networks (CNNs).

Authorized licensed use limited to: The Chinese University of Hong Kong CUHK(Shenzhen). Downloaded on September 15,2025 at 16:58:26 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

Attention-based DNNs are based on the mathematical representation of the attention mechanism, which is a simple mathematical formulation of how the human brain divides its attention. It is a function of n_a key–value pairs $\{\boldsymbol{k}_i, \boldsymbol{v}_i\}_{i=1}^{n_a}$, representing the environment being sensed, and a query vector $\boldsymbol{q}$. The most common form of the attention mechanism is that of scaled dot product attention [30], which processes $\boldsymbol{q}$ and $\{\boldsymbol{k}_i, \boldsymbol{v}_i\}_{i=1}^{n_a}$ into a vector given by

$$\text{Attention}\big(\boldsymbol{q}, \{\boldsymbol{k}_i, \boldsymbol{v}_i\}_{i=1}^{n_a}\big) = \sum_{i=1}^{n_a} \text{softmax}(\text{const} \cdot \boldsymbol{q}^T \boldsymbol{k}_i)\boldsymbol{v}_i.$$

When applied for tasks such as filtering or smoothing, the attention mechanism is used as a trainable layer as a form of self-attention, illustrated in Figure 1(b). A self-attention head is an ML model that applies trained linear layers to map the input sequence in order to obtain the queries, keys, and values, processed via an attention mechanism. A single-head self-attention with parameters $\boldsymbol{\theta} = \{\boldsymbol{W}^q, \boldsymbol{W}^k, \boldsymbol{W}^v\}$ applied for smoothing can be written as $\hat{\boldsymbol{x}}_{1:T} = \mathcal{F}_\theta(\boldsymbol{y}_{1:T})$, where

$$\hat{\boldsymbol{x}}_t = \sum_{\tau=1}^{T} \text{softmax}\big(\text{const} \cdot (\boldsymbol{W}^q \boldsymbol{y}_t)^T (\boldsymbol{W}^k \boldsymbol{y}_\tau)\big)\boldsymbol{W}^v \boldsymbol{y}_\tau. \tag{14}$$

Attention-based DNNs processing time sequences typically apply multiple mappings, as in (14), in parallel as a form of multihead attention. As opposed to RNNs, attention mechanisms do not maintain an internal state vector that is sequentially updated. This makes attention-based DNNs more amenable to parallel training compared with RNNs. However, attention mechanisms, as in (14), are invariant of the order of the processed signal samples and are thus typically combined with additional preprocessing, termed *positional embedding*, that embeds each sample while accounting for its position.

Convolutional neural networks

Unlike RNNs and attention, convolutional neural networks (CNNs) are architectures that originate from image processing, not time sequences. Specifically, CNNs are designed to learn the parameters of spatial kernels, aiming to exploit the locality and spatial stationarity of image data [6, Ch. 9]. Nonetheless, CNNs can also be applied for time sequence processing and particularly for tasks such as filtering and smoothing. For one, the trainable kernel of CNNs can implement a learned finite-impulse response filter (as a form of 1D CNN) and be applied to a time sequence, as demonstrated in Figure 1(c). Alternatively, one can apply CNNs to the time sequence by first converting the time sequence (or an observed window) into the form of an image via, e.g., a short-time Fourier transform and then use a CNN to process this representation.

SS/KF-inspired DNNs

DNNs applied to process time sequences are invariant of the underlying statistics governing the dynamics and learn their operation purely from data. Nonetheless, one can still design DNNs for processing time sequences whose architecture is to some extent inspired by traditional model-based processing of such signals, particularly on SS representations and KF processing.

SS-inspired architectures

A form of DNN architecture that is inspired by SS representation models the filter [and not the dynamics as in (1)], as a deterministic SS model. These ML models, which we term *SS models* (*SSMs*) following [25], parameterize the mapping from $\boldsymbol{y}_t$ into the estimate $\hat{\boldsymbol{x}}_t$ using a latent state vector $\boldsymbol{s}_{t-1}$. Particularly, the architecture operates using two linear mappings, where $\boldsymbol{s}_t$ is first updated via

$$\boldsymbol{s}_t = \boldsymbol{W}^y \boldsymbol{y}_t + \boldsymbol{W}^s \boldsymbol{s}_{t-1}. \tag{15a}$$

Then, the estimate is given by

$$\hat{\boldsymbol{x}}_t = \boldsymbol{W}^x \boldsymbol{s}_t. \tag{15b}$$

The parameters of the resulting ML architecture are $\hat{\boldsymbol{x}}_t = \boldsymbol{W}^x \boldsymbol{s}_t$.

The SSM operation in (15) can be viewed as a form of an RNN, with (15a) implementing (13) and (15b) implementing (12). While the restriction to linear time-invariant learned operators in (15) often results in limited effectiveness, an extension of SSMs, termed *selective SSMs* (see "Selective State-Space Models"), was recently shown to yield efficient and high-performance architectures that are competitive with costly and complex transformers [31].

KF-inspired architectures

While SSMs parameterize a learned filter via an SS representation, one can also design DNN architectures inspired by filters suitable for tracking in SS models, e.g., KF-type algorithms. An example of such architectures is the recurrent Kalman network (RKN), proposed in [7].

The RKN is a DNN whose operation imitates the prediction–update steps of the KF and the EKF. However, while the model-based algorithms realize these computations based on the characterization of the dynamics as an SS model, the RKN parameterizes the predict and update stages as two interconnected DNNs. The predict DNN, whose parameters are $_1$, replaces (4) and computes the a priori state prediction via

$$\hat{\boldsymbol{x}}_{t|t-1}, \boldsymbol{\Sigma}_{t|t-1} = \mathcal{F}_{\theta_1}(\hat{\boldsymbol{x}}_{t-1|t-1}, \boldsymbol{\Sigma}_{t-1|t-1}). \tag{16a}$$

Similarly, the update DNN, parameterized by $\boldsymbol{\theta}_2$, produces the posterior state estimate of (6) via

$$\hat{\boldsymbol{x}}_{t|t}, \boldsymbol{\Sigma}_{t|t} = \mathcal{G}_{\boldsymbol{\theta}_2}(\hat{\boldsymbol{x}}_{t|t-1}, \boldsymbol{\Sigma}_{t|t-1}, \boldsymbol{y}_t). \tag{16b}$$

The resulting architecture can be combined with additional learned input and output processing [7].

Pros and cons of DNN-based state estimation

The DNN architectures detailed so far can be trained to carry out state estimation tasks, i.e., map the observed time sequence into the latent state sequence. Specifically, provided data describing the task, e.g., a labeled dataset composed of trajectories

Authorized licensed use limited to: The Chinese University of Hong Kong CUHK(Shenzhen). Downloaded on September 15,2025 at 16:58:26 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

Selective State-Space Models

An emerging state-space model (SSM) deep neural network, which is the core of the Mamba architecture and its variants [31], is the selective SSM [S2]. Selective SSMs are machine learning models that process time sequences via (15). However, instead of using fixed linear mappings as in (15), here, $\boldsymbol{W}^y, \boldsymbol{W}^s, \boldsymbol{W}^x$ are obtained from learned mappings of the input $\boldsymbol{y}_t$, effectively implementing a time-varying and nonlinear SSM.

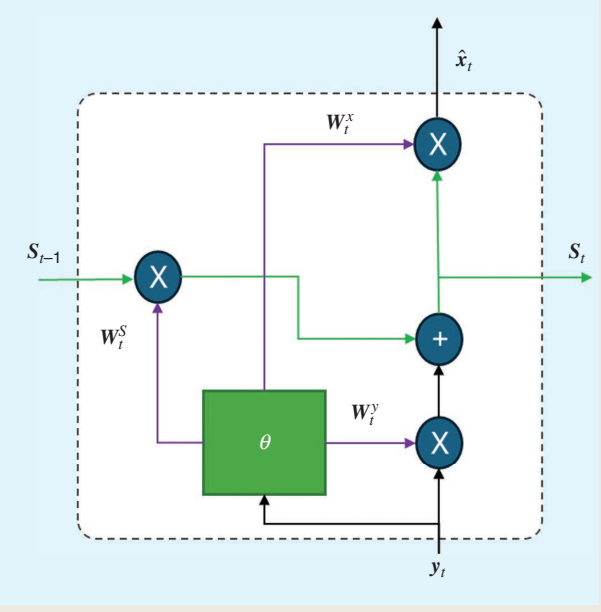

FIGURE S1. The selective SSM.

Generally speaking, consider the discretization with sampling period Δ of continuous-time deterministic SS representations. The resulting state evolution in (15a) is parameterized by

$$\boldsymbol{W}^s = \exp(\Delta \cdot \bar{\boldsymbol{W}}^s) \tag{S2a}$$

$$\boldsymbol{W}^y = (\Delta \cdot \bar{\boldsymbol{W}}^s)^{-1}(\exp(\Delta \cdot \bar{\boldsymbol{W}}^s) - \boldsymbol{I})\Delta \cdot \bar{\boldsymbol{W}}^y \tag{S2b}$$

where $\bar{\boldsymbol{W}}^y, \bar{\boldsymbol{W}}^s$ are the continuous-time state evolution matrices. Selective SSMs implement SSMs based on this representation while using a dedicated layer with parameters $\boldsymbol{\theta}_1$ to map $\boldsymbol{y}_t$ into $\bar{\boldsymbol{W}}^y$, Δ, and $\boldsymbol{W}^x$, where the former two are then substituted in (S2) along with the learned $\bar{\boldsymbol{W}}^s$. The overall parameters are thus $\boldsymbol{\theta} = \{\boldsymbol{\theta}_1, \bar{\boldsymbol{W}}^s\}$. The resulting operation is presented in Figure S1.

Reference

[S2] A. Gu, K. Goel, and C. Ré, "Efficiently modeling long sequences with structured state spaces," in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2022.

of observations and corresponding states, these DNNs learn their mapping from data without relying on any statistical modeling of the dynamics. This form of discriminative learning, i.e., leveraging data to learn to carry out a task end to end [26], excels where model-based methods struggle: it is not affected by inaccurate modeling (thus coping with C1 and C2), and its abstractness allows DNNs to operate reliably in complex settings (C3). In terms of inference speed (C4), while DNNs involve lengthy and complex training, they often provide rapid inference (forward path), particularly when using relatively compact parameterization. This follows, as trained DNNs are highly amenable to parallelization and acceleration of built-in hardware software accelerators, e.g., PyTorch.

Nonetheless, replacing KF-type algorithms with DNNs trained end to end gives rise to various shortcomings, particularly in losing some of the desired properties of model-based methods. For one, DNNs lack in adaptability (P2), as one cannot substitute time-varying parameters of SS models into their operation, and thus, changes may necessitate lengthy retraining. Moreover, DNNs are typically highly parameterized architectures viewed as black boxes that do not share the interpretability of model-based KF-type methods (P3) and are complex to train or even store on limited devices (P5). Furthermore, DNNs struggle in providing uncertainty (P4), for which there is typically no "ground truth" to learn from, and they do not share the theoretical guarantees of KFs (P1).

AI-augmented KFs

The DNN-based approaches discussed so far are highly data driven in the sense that they do not rely on any statistical characterization of the dynamics. Even SS- or KF-inspired architectures, such as the RKN, whose operation generally follows the high-level stages of the KF, are ignorant of any SS modeling.

An alternative approach that aims to benefit from the best of both worlds employs hybrid model-based/data-driven designs via model-based deep learning [11], [33]. Such algorithms typically jointly leverage data along with some form of domain knowledge, i.e., partial knowledge of some components of the underlying SS model (which is often available to some degree, as noted in C1). For state estimation, hybrid algorithms augment KF-type algorithms with deep learning modules, rather than replacing them with DNNs.

As discussed above, the direct application of DNNs reviewed so far is based on task-oriented end-to-end discriminative learning. In the same spirit, existing approaches for augmenting KFs with AI tools can be generally categorized following the ML paradigms of generative and discriminative learning [26].

- *SS-oriented hybrid algorithms*: These use data to learn the underlying statistical model as DNN-aided system identification (thus bearing similarity to generative learning).
- *Task-oriented schemes*: These directly learn to carry out the state estimation task (as a form of discriminative learning)

Authorized licensed use limited to: The Chinese University of Hong Kong CUHK(Shenzhen). Downloaded on September 15,2025 at 16:58:26 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

while leveraging partial state knowledge and principled KF stages as inductive bias.

This categorization, detailed in Figure 2, serves for our review of existing approaches in the sequel.

AI-augmented KFs via task-oriented learning

The first family of AI-aided KFs converts Kalman-type state estimation into an ML architecture that is trainable end to end via DNN augmentation. The key rationale is to leverage deep learning techniques to directly learn the state estimation as a form of discriminative learning. A common approach to designing such architectures is based on using an external DNN, operating alongside a classic state estimator, with recent architectures integrating deep learning modules into the internal processing of Kalman-type algorithms.

External DNN architectures

Utilizing external DNNs aims to enhance KF-type algorithms without altering their internal processing. This approach facilitates design, as one can separate the DNN components from the classic state estimator. Broadly speaking, the leading approaches to utilizing external DNNs employ them either sequentially, i.e., for preprocessing, or in parallel, e.g., as learned correction terms.

Learned preprocessing

A popular approach when dealing with complex measurement models, e.g., visual or multimodal observations, builds on the ability of DNNs to extract meaningful features from complex data. Specifically, it uses a DNN preprocessor to map the observations into a latent space [13], [27].

Architecture

Consider an SS model where the observation model $h_t(\cdot)$ is complex and possibly intractable. In such cases, one can design a DNN with parameters $\boldsymbol{\theta}$ whose output is approximated as obeying a linear Gaussian observations model; i.e.,

$$\boldsymbol{z}_t = \mathcal{F}_\theta(\boldsymbol{y}_t) \approx \boldsymbol{H}\boldsymbol{x}_t + \boldsymbol{w}_t,\ \boldsymbol{w}_t \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{R}) \tag{17}$$

with the observation matrix $\boldsymbol{H}$ being fixed a priori. The latent $\boldsymbol{z}_t$, combined with a known state evolution model, represents observations of a closed-form SS representation. Accordingly, the latent signal can be used as input to a model-based KF-type state estimator to track $\boldsymbol{x}_t$, as described in Figure 3.

Supervised training

In a supervised setting, one has access to a labeled dataset composed of multiple pairs of state and observation length T trajectories; e.g.,

$$\mathbb{D}_s = \{\boldsymbol{x}_{1:T}^{(i)}, \boldsymbol{y}_{1:T}^{(i)}\}_{i=1}^{N}. \tag{18}$$

Given such data, $\boldsymbol{\theta}$ can be trained by encouraging the DNN output to closely match $\boldsymbol{H}\boldsymbol{x}_t$, i.e., using a loss

$$\mathcal{L}_{\mathbb{D}_s}(\boldsymbol{\theta}) = \frac{1}{NT}\sum_{i=1}^{N}\sum_{t=1}^{T}\left\| \mathcal{F}_{\boldsymbol{\theta}}(\boldsymbol{y}_t^{(i)}) - \boldsymbol{H}\boldsymbol{x}_t^{(i)} \right\|_2. \tag{19}$$

When training via (19), the noise covariance $\boldsymbol{R}$ can be estimated empirically from the validation data.

An alternative training approach finds $\boldsymbol{\theta}$ based on its downstream state estimation task, leveraging the differentiable

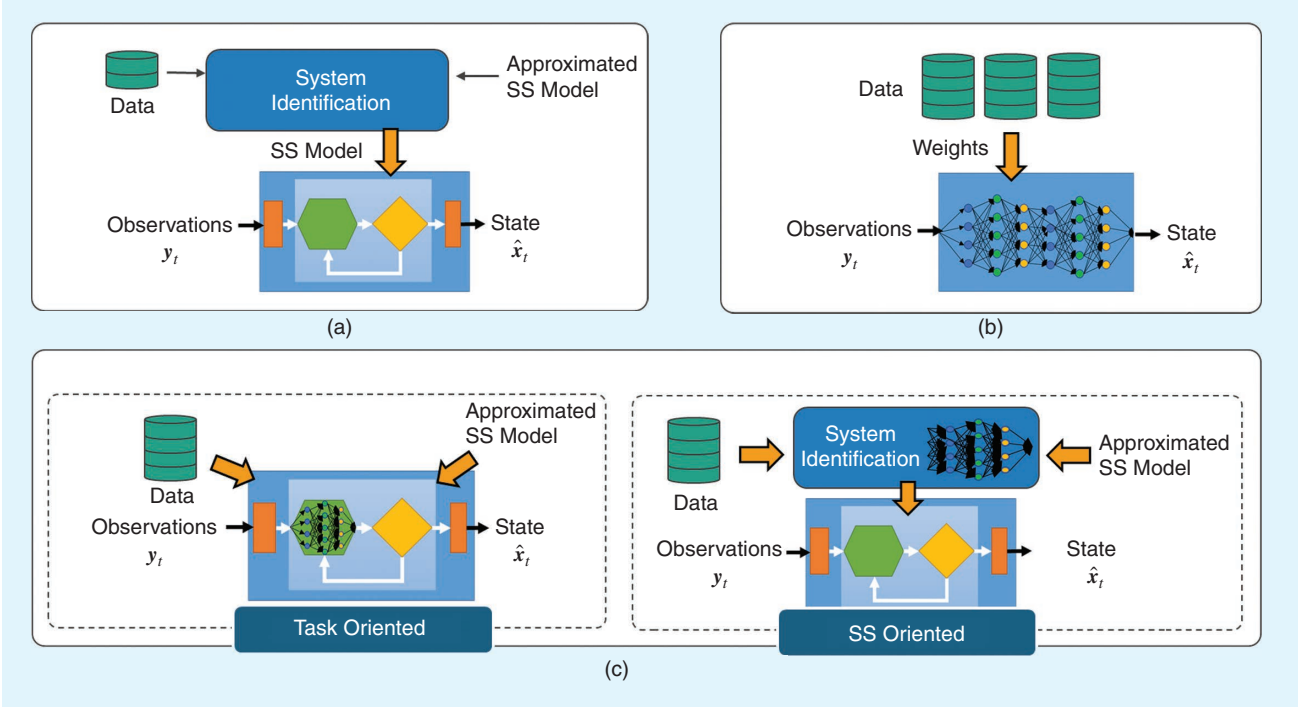

FIGURE 2. A comparison among (a) model-based Kalman-type filters, (b) AI-based filters, and (c) AI-augmented KFs, divided into task-oriented and SS-oriented designs.

Authorized licensed use limited to: The Chinese University of Hong Kong CUHK(Shenzhen). Downloaded on September 15,2025 at 16:58:26 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

operation of KFs [34]. Here, by letting $\hat{\boldsymbol{x}}_t(\mathcal{F}_{\boldsymbol{\theta}}(\boldsymbol{y}_{1:t}))$ be the state estimate obtained by a KF/EKF with observations sequence $\boldsymbol{z}_{1:t}$, where $\boldsymbol{z}_\tau = \mathcal{F}_{\boldsymbol{\theta}}(\boldsymbol{y}_\tau)$, a candidate loss measure is

$$\mathcal{L}_{\mathbb{D}_s}(\boldsymbol{\theta}) = \frac{1}{NT}\sum_{i=1}^{N}\sum_{t=1}^{T}\left\|\hat{\boldsymbol{x}}_t(\mathcal{F}_{\boldsymbol{\theta}}(\boldsymbol{y}_{1:t}^{(i)})) - \boldsymbol{x}_t^{(i)}\right\|_2. \tag{20}$$

The covariance $\boldsymbol{R}$ can be learned during training by being incorporated into the trainable parameters.

Discussion

Using an external preprocessing DNN that is separated from the Kalman-type state estimation algorithm is a design approach geared mostly toward handling complex observation models. As state estimation is carried out by a model-based algorithm, it preserves most of its favorable properties: the state estimation operation is interpretable (P3), and uncertainty measures are provided (P4). The excessive complexity lies in the incorporation of the preprocessing DNN and thus depends on its parameterization as well as on the dimensions of $\boldsymbol{z}_t$. For instance, when $\boldsymbol{z}_t$ is of a much lower dimension compared to $\boldsymbol{y}_t$, the complexity and inference latency savings of applying a KF to $\boldsymbol{z}_t$ compared to using $\boldsymbol{y}_t$ as observations may surpass the added complexity and latency of the preprocessing DNN.

The adaptivity of KFs (P2) is not necessarily preserved. Specifically, when the DNN is fully separate from the state estimator [e.g., when training via (19)], then the state evolution parameters affect only the model-based algorithm, and thus, one can still operate with time-varying $f_t(\cdot)$ (assuming its variations are known). This is not necessarily the case when training via (20), as the latent features are learned to be the ones most supporting state estimation based on the evolution model used during training [17]. Moreover, variations in the observation model typically necessitate retraining of $\boldsymbol{\theta}$.

Using an external preprocessing DNN, while being simple and straightforward to combine with model-based state estimation, relies on Gaussianity and known distribution of the state evolution. The resulting latent SS model is often non-Gaussian, which can impact the tracking accuracy in the latent space, thus not handling C2. Moreover, this approach does not cope with complexities in the state evolution, thus not being geared toward handling inaccuracies (C1) and dominant nonlinearities (C3) in $f_t(\cdot)$.

Learned correction terms

In scenarios with full characterization of the dynamic system as an SS model, such that KF-type algorithms are applicable, though the characterization is not fully accurate, external DNNs can provide correction terms to internal computations of the model-based method [19]. This is illustrated using a representative example based on the augmented Kalman smoother proposed in [19].

Architecture

Consider a linear Gaussian SS model, and focus on the smoothing task, i.e., recovering a sequence of T state variables $\boldsymbol{x}_{1:T}$ from the entire observed sequence $\boldsymbol{y}_{1:T}$. As discussed, when presenting the RTS algorithm, various algorithms exist for such tasks, one of which involves smoothing by seeking to maximize the log-likelihood function via gradient ascent optimization, i.e., by iterating over

$$\boldsymbol{x}_{1:T}^{(q+1)} = \boldsymbol{x}_{1:T}^{(q)} + \gamma\nabla_{\boldsymbol{x}_{1:T}^{(q)}}\log p(\boldsymbol{y}_{1:T}, \boldsymbol{x}_{1:T}^{(q)}) \tag{21}$$

where $\gamma > 0$ is a step-size, for $q = 0, 1, 2, \ldots$ denoting the iteration index. The Markovian nature of the SS model indicates that the gradients in (21) can be computed via message passing, such that for the tth index,

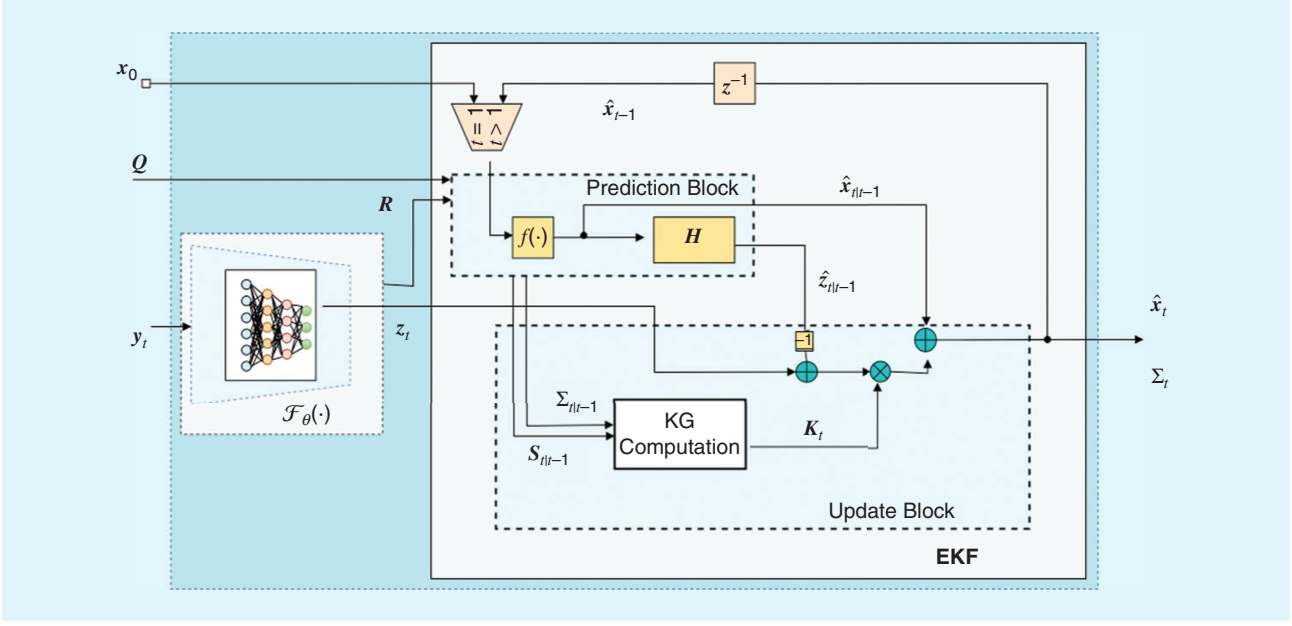

FIGURE 3. An EKF with DNN preprocessing.

Authorized licensed use limited to: The Chinese University of Hong Kong CUHK(Shenzhen). Downloaded on September 15,2025 at 16:58:26 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

$$\nabla_{\boldsymbol{x}_t^{(q)}} \log p(\boldsymbol{y}_{1:T}, \boldsymbol{x}_{1:T}^{(q)}) = \mu_{\boldsymbol{x}_{t-1}^{(q)} \to \boldsymbol{x}_t} + \mu_{\boldsymbol{x}_{t+1}^{(q)} \to \boldsymbol{x}_t} + \mu_{\boldsymbol{y}_t \to \boldsymbol{x}_t} \quad (22)$$

where the summands, referred to as *messages*, are given by

$$\mu_{\boldsymbol{x}_{t-1}^{(q)} \to \boldsymbol{x}_t} = -\boldsymbol{Q}^{-1}(\boldsymbol{x}_t^{(q)} - \boldsymbol{F}\boldsymbol{x}_{t-1}^{(q)}) \quad (23a)$$

$$\mu_{\boldsymbol{x}_{t+1}^{(q)} \to \boldsymbol{x}_t} = \boldsymbol{F}^T\boldsymbol{Q}^{-1}(\boldsymbol{x}_{t+1}^{(q)} - \boldsymbol{F}\boldsymbol{x}_t^{(q)}) \quad (23b)$$

$$\mu_{\boldsymbol{y}_t \to \boldsymbol{x}_t} = \boldsymbol{H}^T\boldsymbol{R}^{-1}(\boldsymbol{y}_t - \boldsymbol{H}\boldsymbol{x}_t^{(q)}). \quad (23c)$$

The iterative procedure in (21) is repeated until convergence, and the resulting $\boldsymbol{x}_{1:T}^{(q)}$ is used as the estimate. In (23), the messages are obtained by assuming a time-invariant version of the SS model in (2) [19].

An external DNN augmentation suggested in [19] enhances the smoother under approximated SS characterization by learning to map the messages in (23) into a correction term $\epsilon_{1:T}^{(q+1)}$, replacing the update rule (21) with

$$\boldsymbol{x}_{1:T}^{(q+1)} = \boldsymbol{x}_{1:T}^{(q)} + \gamma(\nabla_{\boldsymbol{x}_{1:T}^{(q)}} \log p(\boldsymbol{y}_{1:T}, \boldsymbol{x}_{1:T}^{(q)}) + \epsilon_{1:T}^{(q+1)}). \quad (24)$$

Particularly, based on the representation of the smoothing operation as messages exchanged over a Markovian factor graph whose nodes are the state variables, [19] proposed a graph neural network–RNN architecture with parameters $\boldsymbol{\theta}$. This architecture maps the messages in (23) into the correction term $\epsilon_{1:T}^{(q+1)}$; namely,

$$\epsilon_{1:T}^{(q+1)} = \mathcal{F}_{\boldsymbol{\theta}}(\boldsymbol{y}_{1:T}, \{\mu_{\boldsymbol{x}_{t-1}^{(q)} \to \boldsymbol{x}_t}, \mu_{\boldsymbol{x}_{t+1}^{(q)} \to \boldsymbol{x}_t}, \mu_{\boldsymbol{y}_t \to \boldsymbol{x}_t}\}_{t=1}^{T}). \quad (25)$$

Supervised training

The external DNN correction mechanism is trained end to end such that the state predicted by the corrected algorithm best matches the true state. Since smoothing here is based on an iterative algorithm (gradient ascent over the likelihood objective), its training is carried out via a form of deep unfolding [33], fixing the number of iterations to some Q. Then, as the iterative steps in (24) should provide gradually refined state estimates, the loss function accounts for the contribution of the intermediate iterations with a monotonically increasing contribution. Particularly, the loss function used for training $\boldsymbol{\theta}$ is

$$\mathcal{L}_{\mathbb{D}_s}(\boldsymbol{\theta}) = \frac{1}{NT}\sum_{i=1}^{N}\sum_{t=1}^{T}\sum_{q=1}^{Q}\frac{q}{Q}\left\|\hat{\boldsymbol{x}}_t^{(q)}(\boldsymbol{y}_{1:T}^{(i)};\boldsymbol{\theta}) - \boldsymbol{x}_t^{(i)}\right\|_2 \quad (26)$$

where $\hat{\boldsymbol{x}}_t^{(q)}(\boldsymbol{y}_{1:T}^{(i)};\boldsymbol{\theta})$ is the smoothed estimate of $\boldsymbol{x}_t^{(i)}$ produced by the qth iteration, i.e., via (24).

Discussion

Augmenting a KF-type algorithm via a learned correction term is useful when one can still apply the model-based algorithm quite reliably (up to some possible errors that the external DNN corrects). For instance, the augmented algorithm above approximates the dynamic system as a linear Gaussian SS model such that model-based smoothing can be applied based on this formulation, yet it is not MSE optimal (as is the case without discrepancy in the SS model). Consequently, this approach is suitable for tackling C1, yet it is less valid for complex dynamics (C3) and adds only excessive latency to the model-based algorithm (C4).

Integrated DNN architectures

Unlike external architectures, which employ DNNs separately from Kalman-type algorithms, integrated architectures replace intermediate computations with DNNs. Doing so converts the algorithm into a trainable ML model that follows the operation of the classic method as an inductive bias. The key design rationale is to augment computations that depend on missing domain knowledge with dedicated DNNs. Accordingly, existing designs vary based on the absent domain knowledge or, alternatively, on the augmented computation.

Learned KG

A key part of Kalman-type algorithms is the derivation of the KG $\boldsymbol{K}_t$. In particular, its computation via (7) encapsulates the need to propagate the second-order moments of the state and observations. This, in turn, induces the requirement to have full knowledge of the underlying stochasticity (C2), leads to some of the core challenges in dealing with nonlinear SS models (C3), and results in excessive latency, which is associated with propagating these moments (C4). Accordingly, a candidate approach for state estimation in partially known SS models, which is the basis for the KalmanNet algorithm [16] and its variants [17], [35], [36], [37], augments the KG computation with a DNN.

Architecture

Consider a dynamic system represented as an SS model in which one has only a (possibly approximated) model of the time-invariant state evolution function $f(\cdot)$ and the observation function $h(\cdot)$. An EKF with a learned KG (shown in Figure 4) employs a DNN with parameters $\boldsymbol{\theta}$ to compute the KG for each time instant t, denoted $\boldsymbol{K}_t(\boldsymbol{\theta})$. Using this learned KG, an EKF is applied, predicting only the first-order moments via (10) and estimating the state as

$$\hat{\boldsymbol{x}}_t = \hat{\boldsymbol{x}}_{t|t-1} + \boldsymbol{K}_t(\boldsymbol{\theta})(\boldsymbol{y}_t - \hat{\boldsymbol{y}}_{t|t-1}). \quad (27)$$

The architecture does not explicitly track the second-order moments that are needed for the KG, and thus, the learned computation must do so implicitly. Accordingly, the DNN has to 1) process input features that are informative of the noise signals and 2) possess some internal memory capabilities. To meet the first requirement, the input features processed by the DNN typically include 1) differences in the observations, e.g., $\Delta\boldsymbol{y}_t = \boldsymbol{y}_t - \hat{\boldsymbol{y}}_{t|t-1}$, and 2) differences in the estimated state, such as $\Delta\hat{\boldsymbol{x}}_{t-1} = \hat{\boldsymbol{x}}_{t-1} - \hat{\boldsymbol{x}}_{t-1|t-2}$. To provide internal memory, architectures based on RNNs [16], [36] or even transformers [38] can be employed. For instance, a simple FC-RNN-FC architecture was proposed in [16] as well as a more involved interconnection of RNNs, while [36] used two RNNs—one for

Authorized licensed use limited to: The Chinese University of Hong Kong CUHK(Shenzhen). Downloaded on September 15,2025 at 16:58:26 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

tracking $\Sigma_{t|t-1}$ and one for tracking $\boldsymbol{S}_{t|t-1}^{-1}$—combined into the KG via (7).

While the above architecture is formulated for filtering, it extends to smoothing, particularly when smoothing via the RTS algorithm [5] employs a KF for a forward pass, followed by an additional backward pass (8) that uses the backward gain matrix $\tilde{\boldsymbol{K}}_t$ given in (9). Consequently, the DNN-augmented methodology above extends to smoothing by introducing an additional KG DNN that learns to compute $\tilde{\boldsymbol{K}}_t$. Moreover, the single forward–backward pass of the RTS algorithm is not necessarily MSE optimal in nonlinear SS models, the learned RTS can be applied in multiple iterations, using the estimates produced at a given forward–backward pass be used as the input to the following pass [35]. Here, the fact that each pass is converted into an ML architecture allows learning different forward and backward gain DNNs for each pass as a form of deep unfolding [33]

Training

KalmanNet and its variants use DNNs to compute the KG. While there is no "ground truth" KG when deviating from linear Gaussian SS models, the overall augmented state estimation algorithm is trainable in a supervised manner. Particularly, given a labeled dataset as in (18), a candidate loss measure is

$$\mathcal{L}_{\mathbb{D}_s}(\boldsymbol{\theta}) = \frac{1}{NT}\sum_{i=1}^{N}\sum_{t=1}^{T}\left\|\hat{\boldsymbol{x}}_t(\boldsymbol{y}_{1:t}^{(i)};\boldsymbol{\theta}) - \boldsymbol{x}_t^{(i)}\right\|_2 \tag{28}$$

where $\hat{\boldsymbol{x}}_t(y_{1:t};\boldsymbol{\theta})$ is the state estimated using the augmented EKF with parameters $\boldsymbol{\theta}$ and observations $\boldsymbol{y}_{1:t}$.

While KalmanNet is designed to be trained from labeled data via (28), in some settings, it can also be trained without providing it with ground truth state labels. This can be achieved via the following approaches:

1) *Observation prediction*: The fact that the DNN-augmented algorithm preserves the interpretable operation of Kalman-type tracking can be leveraged for unsupervised learning, i.e., learning from data of the form

$$\mathbb{D}_u = \{\boldsymbol{y}_{1:T}^{(i)}\}_{i=1}^{N}. \tag{29}$$

Specifically, as Kalman-type algorithms with time-invariant SS models internally predict the next observation as $\hat{\boldsymbol{y}}_{t+1|t} = h(f(\hat{\boldsymbol{x}}_t))$ in (5), a possible unsupervised loss is [21]

$$\mathcal{L}_{\mathbb{D}_u}(\boldsymbol{\theta}) = \frac{1}{NT}\sum_{i=1}^{N}\sum_{t=0}^{T-1}\left\|h(f(\hat{\boldsymbol{x}}_t(\boldsymbol{y}_{1:t}^{(i)};\boldsymbol{\theta}))) - \boldsymbol{y}_{t+1}^{(i)}\right\|_2. \tag{30}$$

2) *Downstream task*: Often, in practice, state estimation is carried out as part of an overall processing chain, with some downstream tasks. In various applications, one can evaluate an estimated state without requiring a ground truth value. When such evaluation is written (or approximated) as a function that is differentiable with respect to the estimated state, it can be used as a training loss. This approach was shown to enable unsupervised learning of KalmanNet when integrated in stochastic control systems, where it is combined with a linear quadratic regulator [39], as well as in financial pairs trading, where the states tracked are financial features used for trading policies [24].

Discussion

The design of DNN-aided Kalman-type algorithms by augmenting the KG computation is particularly suitable for tackling identified C1–C4. Unlike purely end-to-end DNNs designed for generic processing of time sequences, augmenting the KG allows leveraging domain knowledge of the state evolution and observation models, as these are utilized in the prediction step. However, as the following processing of these predictions utilizes a DNN that is trained based on the accuracy of the overall algorithm, mismatches and approximation errors are learned to be corrected, thus fully tackling C1. As the DNN augmentation bypasses the need to track second-order moments, the resulting algorithm does not require knowledge of the distribution of the noises. In fact, the resulting filter is not linear (as the KG depends on the observations), allowing learning nonlinear state estimators that are suitable for non-Gaussian SS models (C2) and nonlinear dynamics (C3). Finally, as the computation of the KG and the propagation of the second-order moments induces most of the latency in

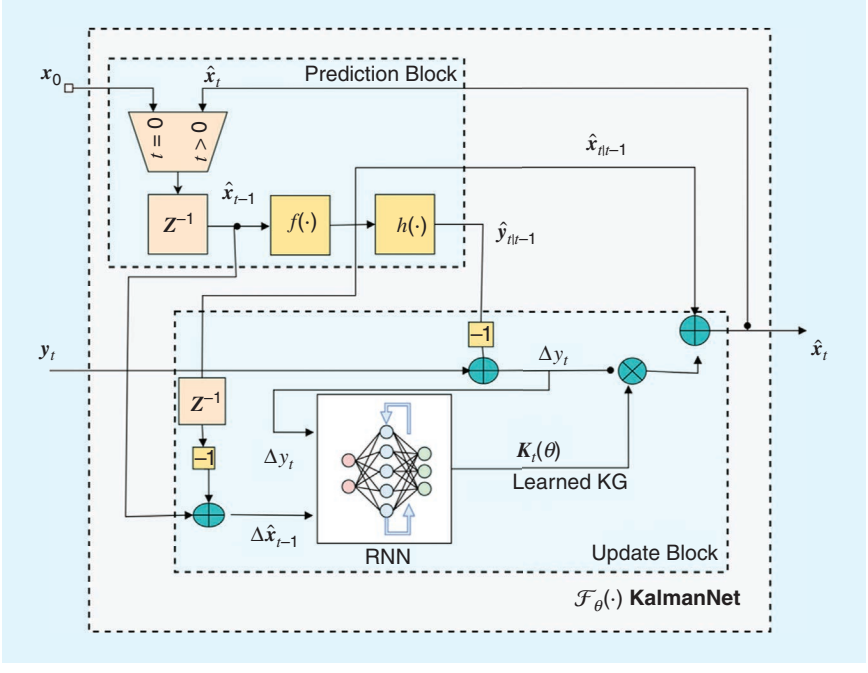

FIGURE 4. An EKF with learned KG.

Authorized licensed use limited to: The Chinese University of Hong Kong CUHK(Shenzhen). Downloaded on September 15,2025 at 16:58:26 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

Uncertainty Extraction From Learned Kalman Gain

Kalman-type algorithms use both the prior covariance $\Sigma_{t|t-1}$ and the Kalman gain (KG) to compute the posterior error covariance Σ_t, which represents the estimation uncertainty. Particularly, by (6b) and (7), the extended Kalman filter (EKF) computes the error covariance as

$$\Sigma_t = (\boldsymbol{I} - \boldsymbol{K}_t\hat{\boldsymbol{H}}_t)\Sigma_{t|t-1}. \tag{S3a}$$

Consequently, despite the fact that artificial intelligence-aided filters with learned KG do not explicitly track the second-order moments, in some cases, these can still be recovered via (S3a), as shown in [40].

In particular, the architecture provides the learned KG as $\boldsymbol{K}_t(\boldsymbol{\theta})$. Accordingly, when the matrix $\tilde{\boldsymbol{H}}_t = (\hat{\boldsymbol{H}}_t^{\top}\hat{\boldsymbol{H}}_t)^{-1}$ exists, then, combining (5b) and (7), the prior covariance can be recovered via

$$\Sigma_{t|t-1} = (\boldsymbol{I} - \boldsymbol{K}_t(\boldsymbol{\theta})\hat{\boldsymbol{H}}_t)^{-1}\boldsymbol{K}_t(\boldsymbol{\theta})\boldsymbol{R}\hat{\boldsymbol{H}}_t\tilde{\boldsymbol{H}}_t. \tag{S3b}$$

This indicates that when one accurately computes $\hat{\boldsymbol{H}}_t$ [via (11)] and has knowledge of the measurement noise covariance $\boldsymbol{R}$, then the learned KG can be used to recover the error covariance via (S3a) and (S3b), as illustrated in Figure S2. The extracted covariance can be encouraged to match the empirical estimation error in training by, e.g., adding an additional loss term that penalizes uncertainty extraction or by imposing a Gaussian prior on the error and minimizing its likelihood; see [40].

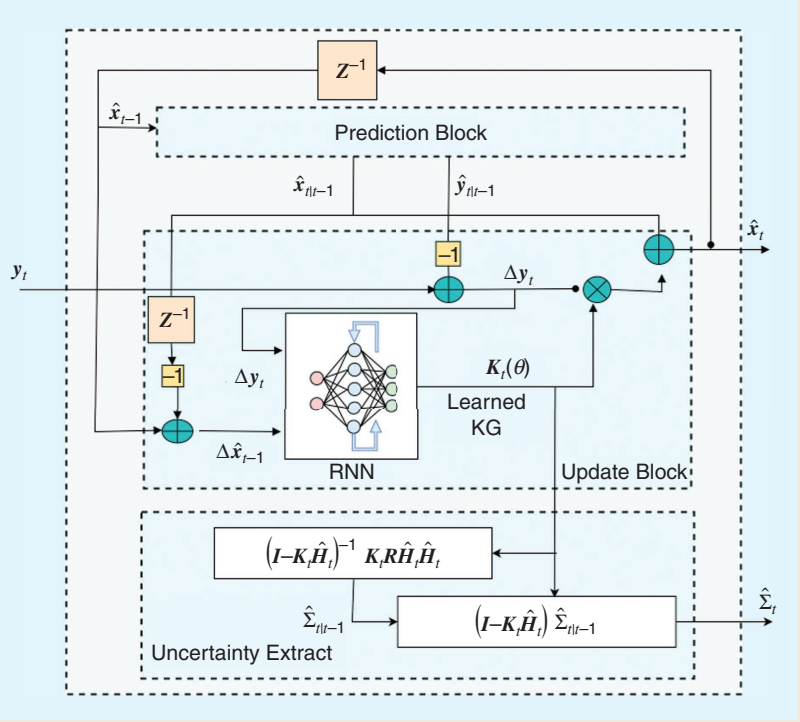

FIGURE S2. The EKF with learned KG and covariance extraction. RNN: recurrent neural network.

Kalman-type algorithms for nonlinear SS models, replacing these computations with a compact DNN often leads to more rapid inference (C4) [35].

The fact that the resulting state estimator is converted into a trainable discriminative ML architecture makes it natural to be combined with DNN-based preprocessing (as in Figure 3), e.g., for processing images or high-dimensional data. Particularly, the learned state estimator can be trained jointly with the preprocessing stage, thus having it learn latent features that are most useful for processing with the learned filter, without requiring one to approximate the distribution of the latent measurement noise [17].

Unlike state estimation based on end-to-end DNNs, which also tackles C1–C4 (as discussed in the previous section), the usage of Kalman-type algorithms as an inductive bias also allows preserving some of the desirable properties of model-based state estimators. In particular, the interpretable operation is preserved in the sense that the internal features are associated with concrete meaning (P3), which can be exploited for, e.g., unsupervised learning. Moreover, while the error covariance is not explicitly tracked, in some cases, it can actually be recovered from the learned KG (see "Uncertainty Extraction From Learned Kalman Gain"), thus providing P4 to some extent. In addition, the fact that only an internal computation is learned allows utilizing relatively compact DNNs, striking a balance between the excessive complexity of end-to-end DNNs and the relatively low one of model-based methods (P5).

The core limitations of designing an AI-aided state estimator by augmenting the KG computations are associated with the state estimator's adaptivity. The design is particularly geared toward time-invariant state evolution and observation functions. As the KG computation is coupled with these models, any deviation, even known ones, is expected to necessitate retraining. This can be alleviated under some forms of variations using hypernetworks [41], i.e., having an additional DNN that updates the KG DNN, while inducing some complexity increase. Moreover, the architecture is designed for supervised learning. The ability to train in an unsupervised manner via (30) relies on full knowledge of $f(\cdot)$ and $h(\cdot)$, thus not being applicable in the same range of settings as its supervised counterpart. The same also holds for uncertainty extraction via (S3), which requires some additional domain knowledge compared to that needed for merely tracking the state via (27).

Authorized licensed use limited to: The Chinese University of Hong Kong CUHK(Shenzhen). Downloaded on September 15,2025 at 16:58:26 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

The introduced learned KG algorithms calculate the complete gain by a DNN. However, if a Kalman-type filter, such as the UKF or divided-difference filter, depends on a user-defined scaling parameter, a DNN can be used for the parameter prediction (instead of the complete gain calculation) [42]. As a consequence, such a DNN-reasoned scaling parameter affects all elements of the KG and is able, up to a certain extent, to compensate for model discrepancies or linearization errors. This DNN-augmented filter inherently provides the estimate error covariance matrix but under the assumption of the known SS model (3).

Learned state estimation

Another class of integrated DNN architectures includes methods that learn the task of state estimation in a data-driven manner using the known observation model without any knowledge of the state evolution model. The state evolution model in (1a) may not be linear or require $\boldsymbol{v}_t$ to be Gaussian noise. A candidate approach in this category is the data-driven nonlinear state estimation (DANSE) method that provides a closed-form posterior of the underlying state using linear state measurements under Gaussian noise [20]. The noisy measurements follow the linear observation model similar to (2b); namely,

$$\boldsymbol{y}_t = \boldsymbol{H}_t \boldsymbol{x}_t + \boldsymbol{w}_t \tag{31}$$

where the measurement noise is $\boldsymbol{w}_t \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{R})$ with known covariance $\boldsymbol{R}$. The matrix $\boldsymbol{H}_t$ is assumed to be full column rank and have a known $\forall t$. Note that $p(\boldsymbol{y}_t \,|\, \boldsymbol{x}_t) = \mathcal{N}(\boldsymbol{H}_t \boldsymbol{x}_t, \boldsymbol{R})$.

Architecture

The core of the DANSE method relies on the sequential modeling capability of RNNs [6, Ch. 10]. DANSE consists of an RNN with parameters $\boldsymbol{\theta}$ that parameterizes a Gaussian prior on $\boldsymbol{x}_t$ given $\boldsymbol{y}_{1:t-1}$ at a given time t. Concretely, the prior distribution $p(\boldsymbol{x}_t \,|\, \boldsymbol{y}_{1:t-1}; \boldsymbol{\theta})$ is

$$\begin{aligned} p(\boldsymbol{x}_t \,|\, \boldsymbol{y}_{1:t-1}; \boldsymbol{\theta}) &= \mathcal{N}(\hat{\boldsymbol{x}}_{t|t-1}(\boldsymbol{\theta}), \Sigma_{t|t-1}(\boldsymbol{\theta})) \\ \text{s.t.}\{\hat{\boldsymbol{x}}_{t|t-1}(\boldsymbol{\theta}), \Sigma_{t|t-1}(\boldsymbol{\theta})\} &= \mathcal{F}_{\boldsymbol{\theta}}(\boldsymbol{y}_{t-1}) \end{aligned} \tag{32}$$

where $\mathcal{F}_{\boldsymbol{\theta}}(\boldsymbol{y}_{t-1})$ refers to the RNN that recursively processes the sequence of past observations $\boldsymbol{y}_{1:t-1}$, as described in (12) and (13). Also, $\hat{\boldsymbol{x}}_{t|t-1}(\boldsymbol{\theta})$ and $\Sigma_{t|t-1}(\boldsymbol{\theta})$ denote the mean vector and the covariance matrix, respectively, of the RNN-based Gaussian prior at time t. The covariance matrix $\Sigma_{t|t-1}(\boldsymbol{\theta})$ can be designed to be full or diagonal. More accurately, the hidden state of the RNN in (32) at time t is nonlinearly transformed using feedforward networks with appropriate activation functions to obtain $\hat{\boldsymbol{x}}_{t|t-1}(\boldsymbol{\theta})$ and $\Sigma_{t|t-1}(\boldsymbol{\theta})$ [20]. In [20], DANSE uses a GRU for implementation purposes, owing to its simplicity and modeling capability.

Using the fact that the measurement system in (31) is linear and also Gaussian, we have $p(\boldsymbol{y}_t \,|\, \boldsymbol{x}_t) = \mathcal{N}(\boldsymbol{H}_t \boldsymbol{x}_t, \boldsymbol{R})$. This allows the use of the representation in equation (S1) to obtain $p(\boldsymbol{x}_t \,|\, \boldsymbol{y}_{1:t}; \boldsymbol{\theta})$ in closed form [20]. One can show that the posterior distribution holds $p(\boldsymbol{x}_t \,|\, \boldsymbol{y}_{1:t}; \boldsymbol{\theta}) = \mathcal{N}(\hat{\boldsymbol{x}}_{t|t}(\boldsymbol{\theta}), \Sigma_{,t|t}(\boldsymbol{\theta}))$, with

$$\begin{aligned} \hat{\boldsymbol{x}}_{t|t}(\boldsymbol{\theta}) &= \hat{\boldsymbol{x}}_{t|t-1}(\boldsymbol{\theta}) + \boldsymbol{K}'_t(\boldsymbol{\theta}) \Delta \boldsymbol{y}_t(\boldsymbol{\theta}), \\ \Sigma_{t|t}(\boldsymbol{\theta}) &= \Sigma_{t|t-1}(\boldsymbol{\theta}) - \boldsymbol{K}'_t(\boldsymbol{\theta}) \boldsymbol{S}_{t|t-1}(\boldsymbol{\theta}) (\boldsymbol{K}'_t(\boldsymbol{\theta}))^{\mathsf{T}}. \end{aligned} \tag{33}$$

Data-Driven Nonlinear State Estimation Architecture

The data-driven nonlinear state estimation (DANSE) method utilizes a recurrent neural network (RNN) for recursively processing the input sequence $\boldsymbol{y}_{1:t-1}$ [20]. An example of this sequential processing at time t is provided in Figure S3. One can use prominent RNNs for sequential processing, such as gated recurrent units (GRUs) or long short-term memory. In DANSE, a GRU was used due to simplicity and good expressive power compared to vanilla RNNs. Specifically, for modeling the mean vector $\hat{\boldsymbol{x}}_{t|t-1}(\boldsymbol{\theta})$ and diagonal covariance matrix $\Sigma_{t|t-1}(\boldsymbol{\theta})$ of the parameterized Gaussian prior in (32), the hidden state of the GRU was nonlinearly transformed using feedforward networks with suitable activation functions. In Figure S3, the "Accumulator" block is shown for ease of understanding; in practice, the block is embedded in the recursive operation of the RNN.

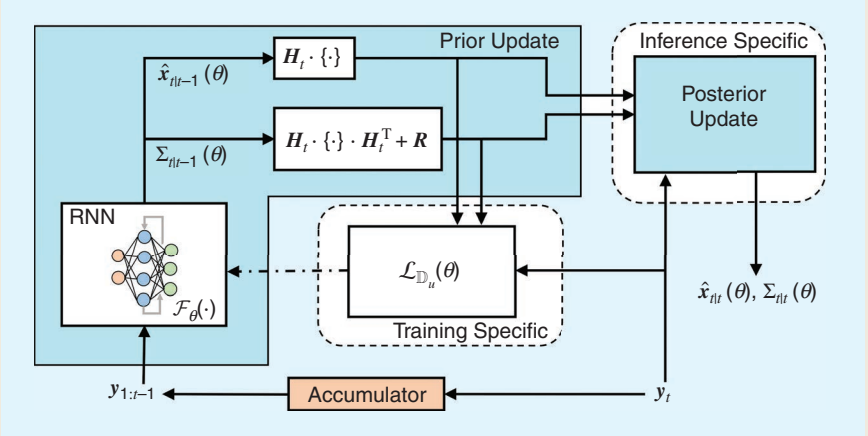

FIGURE S3. DANSE at time t, highlighting both training-specific and inference-specific blocks [20]. Dashed lines represent the gradient flow to $\mathcal{F}_\theta$ during training, and solid lines indicate the information flow during training/inference.

Authorized licensed use limited to: The Chinese University of Hong Kong CUHK(Shenzhen). Downloaded on September 15,2025 at 16:58:26 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

Notably, (33) resembles the Kalman update equations in (6) and (7), where

$$\begin{aligned}\boldsymbol{K}_t'(\boldsymbol{\theta}) &\triangleq \Sigma_{t|t-1}(\boldsymbol{\theta})\boldsymbol{H}_t^\top \boldsymbol{S}_{t|t-1}^{-1}(\boldsymbol{\theta})\\ \boldsymbol{S}_{t|t-1}(\boldsymbol{\theta}) &\triangleq \boldsymbol{H}_t\Sigma_{t|t-1}(\boldsymbol{\theta})\boldsymbol{H}_t^\top + \boldsymbol{R}\\ \Delta\boldsymbol{y}_t(\boldsymbol{\theta}) &\triangleq \boldsymbol{y}_t - \boldsymbol{H}_t\hat{\boldsymbol{x}}_{t|t-1}(\boldsymbol{\theta}).\end{aligned} \tag{34}$$

Note that $\boldsymbol{K}_t'(\boldsymbol{\theta})$ is conceptually similar as the traditional KG term in (7) but computed in a data-driven manner and that the computation approach is different from the one in Figure 4, due to (32).

In the backdrop of the KF, a crucial aspect of DANSE is that there is no Gaussian propagation of the posterior in (33) to the prior because DANSE does not use any state evolution model, such as (1a). The parameters $\hat{\boldsymbol{x}}_{t|t-1}(\boldsymbol{\theta})$, $\Sigma_{t|t-1}(\boldsymbol{\theta})$ of the prior are obtained directly from the RNN, which does not use any explicit state evolution model as in (1a) or require any knowledge regarding the same, e.g., first-order Markovian, Gaussian process noise, and so on. A simplified schematic of DANSE appears in Figure 5, with a detailed schematic and details regarding architectural choices present in "Data-Driven Nonlinear State Estimation Architecture."

Training

The parameters $\boldsymbol{\theta}$ in DANSE are learned in an unsupervised manner using a training dataset consisting of only noisy measurement trajectories $\mathbb{D}_u = \{\boldsymbol{y}_{1:T}^{(i)}\}_{i=1}^N$, where N is the number of training samples and every ith sample is assumed to have the same trajectory length T. The learning mechanism is based on maximizing the likelihood of the dataset $\mathbb{D}$, where one computes the joint likelihood of a full sequence $\boldsymbol{y}_{1:T}$. To achieve this, one first calculates the conditional marginal distribution $p(\boldsymbol{y}_t|\boldsymbol{y}_{1:t-1};\boldsymbol{\theta})$ using the Chapman–Kolmogorov equation (S1), as follows:

$$\begin{aligned}p(\boldsymbol{y}_t|\boldsymbol{y}_{1:t-1};\boldsymbol{\theta}) &= \int p(\boldsymbol{y}_t|\boldsymbol{x}_t)p(\boldsymbol{x}_t|\boldsymbol{y}_{1:t-1};\boldsymbol{\theta})d\boldsymbol{x}_t\\ &= \mathcal{N}(\boldsymbol{H}_t\hat{\boldsymbol{x}}_{t|t-1}(\boldsymbol{\theta}),\, \boldsymbol{H}_t\Sigma_{t|t-1}(\boldsymbol{\theta})\boldsymbol{H}_t^\top + \boldsymbol{R})\\ &= \mathcal{N}(\boldsymbol{H}_t\hat{\boldsymbol{x}}_{t|t-1}(\boldsymbol{\theta}),\, \boldsymbol{S}_{t|t-1}(\boldsymbol{\theta}))\end{aligned} \tag{35}$$

where $\boldsymbol{S}_{t|t-1}(\theta)$ is defined in (34). Then for the sequence $\boldsymbol{y}_{1:T}$, one can calculate the joint likelihood as

$$p(\boldsymbol{y}_{1:T};\boldsymbol{\theta}) = \prod_{t=1}^{T} p(\boldsymbol{y}_t|\boldsymbol{y}_{1:t-1};\boldsymbol{\theta}). \tag{36}$$

Thus, using $\mathbb{D}_u$, (35), and (36), the optimization problem can be formulated as maximization of the logarithm of the joint log-likelihood of $\mathbb{D}_u$, as follows:

$$\max_{\boldsymbol{\theta}} \log \prod_{i=1}^{N}\prod_{t=1}^{T} p(\boldsymbol{y}_t^{(i)}|\boldsymbol{y}_{1:t-1}^{(i)};\boldsymbol{\theta}) = \min_{\boldsymbol{\theta}} \mathcal{L}_{\mathbb{D}_u}(\boldsymbol{\theta}) \tag{37}$$

where the loss term $\mathcal{L}_{\mathbb{D}_u}(\theta)$ is obtained using (35), as follows:

$$\begin{aligned}\mathcal{L}_{\mathbb{D}_u}(\boldsymbol{\theta}) = \sum_{i=1}^{N}\sum_{t=1}^{T}\Big\{&\frac{n}{2}\log 2\pi + \frac{1}{2}\log\det(\boldsymbol{S}_{t|t-1}^{(i)}(\boldsymbol{\theta}))\\ &+ \frac{1}{2}\Big\|\boldsymbol{y}_t - \boldsymbol{H}_t\hat{\boldsymbol{x}}_{t|t-1}^{(i)}(\boldsymbol{\theta})\Big\|^2_{(S_{t|t-1}^{(i)}(\boldsymbol{\theta}))^{-1}}\Big\}.\end{aligned} \tag{38}$$

In (38), $\{\hat{\boldsymbol{x}}_{t|t-1}^{(i)}(\boldsymbol{\theta}), \Sigma_{t|t-1}^{(i)}(\boldsymbol{\theta})\}$ are the Gaussian prior parameters (32) obtained using the ith sample $\boldsymbol{y}_{1:t-1}^{(i)}$ as input at time t. The parameters $\boldsymbol{\theta}$ are thus learned by minimizing $\mathcal{L}_{\mathbb{D}_u}(\boldsymbol{\theta})$ with respect to $\boldsymbol{\theta}$ in (37). Practically, this is achieved by using minibatch stochastic gradient descent [20, Sec. II-C].

Discussion

The DANSE approach can address the challenges associated with model-based KF-type algorithms explained in C1–C4. DANSE partly addresses C1, as it does not require knowledge of the state evolution model (1a) or the involved process noise in (1a). However, it requires knowledge of the linear observation model shown in (31), where $\boldsymbol{H}_t$ should be full rank and noise covariance $\boldsymbol{R}$ should be known a priori. The parameterization of the Gaussian prior in (32) ensures that DANSE can capture long-term temporal dependency in the state model so that one is not constrained to state estimation for Markovian SS models. This is also mentioned [20], where DANSE is introduced in a "model-free process" setting. In [20], the authors define a "model-free process" as a process where the knowledge of the state evolution model is absent. This also partly tackles C2 since there is no requirement on the distribution of $\boldsymbol{v}_t$ in (1a) to be Gaussian. At the same time, it is required that $\boldsymbol{w}_t$ be Gaussian with a fixed covariance $\boldsymbol{R}$, as shown in (31).

The use of RNNs and the parameterization of the Gaussian prior also ensures that one can learn a nonlinear state estimation method for nonlinear SS models similar to KalmanNet. This in turn helps to tackle the challenge described in C3. The training of DANSE, as explained in the previous paragraph, is unsupervised and offline, as it requires access to a dataset $\mathbb{D}_u$ consisting of noisy measurements. Once the training is completed, the inference step in DANSE is causal and separated from the offline training step. This ensures that one has a rapid inference at test time. This helps address C4, as DANSE does not require any linearization of the state evolution model as in the EKF or sampling sigma points or particles in the UKF or PF, respectively.

DANSE is also partly interpretable, as its posterior update in (33) is tractable and bears similarity to that of the KF (6). This is ensured by using the linear observation model in (31) and the choice of the Gaussian prior for $\boldsymbol{x}_t$ in (32). This ensures tractable posterior updates, resonating with the feature of P3 regarding the interpretability of conventional model-based approaches using first- and second-order moments. Furthermore, the advantage of P4 regarding providing uncertainty

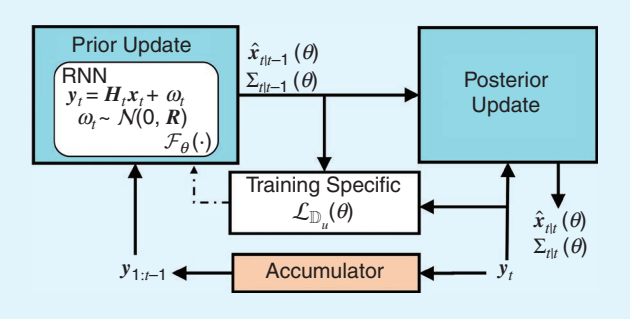

FIGURE 5. A simplified schematic of DANSE [20].

Authorized licensed use limited to: The Chinese University of Hong Kong CUHK(Shenzhen). Downloaded on September 15,2025 at 16:58:26 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

estimates is also present in DANSE, as we have both prior and posterior covariance estimates in (32) and (33). As mentioned earlier, the key difference compared to model-based approaches is the absence of propagating the posterior moments to the prior at the next time point.

Finally, it is worth noting that DANSE cannot immediately adapt to underlying changes in the SS model and would require retraining. This also applies to changes in the observation model, as it requires complete knowledge of the same. Hence, DANSE does not have the advantage of P2, which is inherent in model-based approaches and AI-aided approaches that utilize the knowledge of the state evolution model or additional hypernetworks.

AI-augmented KFs via SS-oriented learning

The second family of AI-aided KFs uses data to learn, refine, or augment the underlying statistical model using deep learning tools. Then, the data-augmented SS can be directly used in Kalman-type state estimation. Instead of learning the state estimation or the specific parameters in the KF, e.g., KG, the key rationale of SS-oriented learning is to exploit the data to obtain a more accurate model. At the same time, SS-oriented learning preserves explainability of the state and brings associated benefits of the statistical estimation,[1] such as the inherent calculation of the covariance matrix of the estimate error [15].

Architecture

The SS-oriented AI-augmented KFs generally focus on data-augmented modeling of the state and measurement equations [43], [44]. Some approaches [45], [46], [47], [48], [49] assume the measurement model to be known, for which the motivation is twofold: 1) the sensors often can be well modeled based on first principles, but a state dynamics model is typically approximate and depends on a user decision (for example, the object kinematic can be modeled by a nearly constant velocity/acceleration model or a Singer model) [15], and 2) there is a need to estimate the state carrying physical meaning, i.e., to preserve interpretability of the state estimate. Interpretability may be lost when both equations are modeled with data augmentation. For convenience and for compact exposition, the following text focuses on data-augmented modeling of the state dynamics. The concept of data-augmented modeling thus resides in the definition of four models of state dynamics.

1) A true model (TM), or a data generator, is a complex and context-dependent model that cannot be expressed in a finite-dimensional form. The TM is thus more of a theoretical construct.
2) A physics-based model (PBM) is defined by (3a) and can be understood as the "best achievable" model of the given complexity found from the first principles.
3) A data-driven model (DDM) is solely identified from data. The use of ML in system identification has been studied for several decades [50], and the DDM can be written as

$$\boldsymbol{x}_{t+1} = g(\boldsymbol{x}_t; \boldsymbol{\theta}^{\mathrm{DNN}}) + \boldsymbol{v}_t^{\mathrm{DDM}} \tag{39}$$

where $\boldsymbol{v}_t^{\mathrm{DDM}}$ is a process noise and $g(\cdot)$ is a vector-valued function, possibly a DNN, which is parameterized by the vector $\boldsymbol{\theta}^{\mathrm{DNN}}$ designed to minimize the discrepancy between outputs of (39) and the TM.

4) The current trend is to consider models that are not purely trained from data, as in DDMs, making use of the available information that PBMs offer. These hybrid models are discussed in more detail in the sequel.

The approaches to data-augmented modeling of the state equation can be classified in terms of the SS components that the DNN characterizes or is embedded in. In the general SS structures [see Figure 6(a)], the whole model of state dynamics is identified from data and replaced by a DNN[2] (39). In [45], FC DNNs were used to represent the function $f_t(\cdot)$, and in [43], the authors use predictive partial autoencoders to find the state and its mapping to the measurement. In [44], FC DNNs trained by the expectation–maximization algorithm were used to represent both functions. A comparison of the performance of several general SS structures was presented in [14], considering LSTM, GRUs, and variational autoencoders (VAEs). The authors assumed Gaussian likelihood and combined the RNN and VAE into a deep SS model. Stochastic RNNs were introduced for system identification in [46] to account for stochastic properties of the SS model. A similar approach follows physics-informed neural networks (PINNs) [47], modeling the state equation by a DNN for which training is constrained by the physics describing the PBM [see Figure 6(b)].

Another approach to model the state equation is to consider a fixed structure, such as a linear parameter-varying model (2a), and use the DNNs to represent the matrices [51] [see Figure 6(c)]. Such an approach has also been adopted in [34], where the covariance matrices $\boldsymbol{Q}$ and $\boldsymbol{R}$ were provided by a DNN.

A similar idea was elaborated in [48], where an incremental SS structure was proposed to separate $f_t(\cdot)$ into linear and nonlinear parts and use a DNN to represent the nonlinear part [see Figure 6(d)]. In [49], a hybrid model [see Figure 6(e)] was proposed, which combines the PBM for some state elements and the DNN for the rest. It leverages the fact that the dynamic of some states is precisely known (e.g., the position derivative is velocity).

The data-augmented modeling represented by the data-augmented PBM (APBM) combines the physics- and data-driven components into a single model, which reads

$$\boldsymbol{x}_{t+1} = g\big(f_t(\boldsymbol{x}_t), \boldsymbol{x}_t, \boldsymbol{d}_t; \boldsymbol{\theta}_t^{\mathrm{APBM}}\big) + \boldsymbol{v}_t^{\mathrm{APBM}} \tag{40}$$

where $\boldsymbol{v}_t^{\mathrm{APBM}}$ is a process noise, d_t represents additional data,[3] the function $g(\cdot)$ is aware of the PBM part, and the vector

[1] Such an augmented model can also be directly used for designing a fault detection or control algorithm.

[2] A structure similar to the general one can be considered for the continuous-time models, where the DNN represents the function in the differential state equation and then the Euler discretization is used to obtain a discrete-time model used for the estimation.

[3] These denote data related to the system available to the user but not used in the PBM as additional inputs to avoid overly complex models (e.g., ionospheric models and weather forecasts).

Authorized licensed use limited to: The Chinese University of Hong Kong CUHK(Shenzhen). Downloaded on September 15,2025 at 16:58:26 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

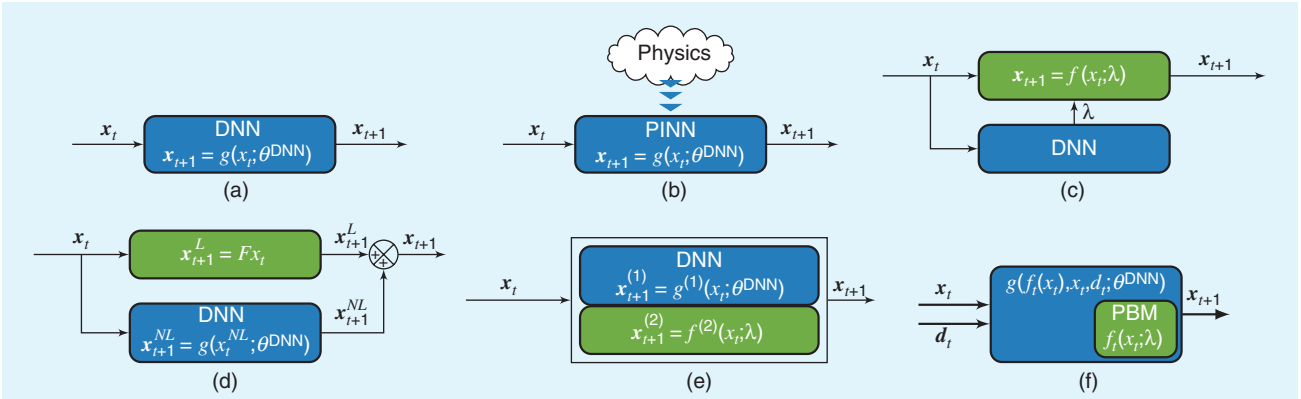

FIGURE 6. Structures for SS-oriented learning: the (a) general SS structure, (b) physics-informed SS structure, (c) fixed SS structure, (d) incremental SS structure, (e) hybrid physics-based SS structure, and (f) data-augmented PBM (APBM) SS structure. PINN: physics-informed neural network.

$\boldsymbol{\theta}_t^{\text{APBM}}$ is designed to minimize the discrepancy between outputs of (40) and the TM. The APBM compensates the PBM structure and parameter mismodeling using information extracted from available data. Consequently, the APBM preserves the physical meaning of the state components and exploits actual system behavior dependencies, which were ignored in the PBM design. An example of this modeling versatility was shown in [52], where APBMs were used to filter a high-order Markov process without the need for order selection. One important characteristic of the APBM formulation (40) is the flexibility to cope with the nonstationarity of the model over time or space ($\boldsymbol{\theta}_{t-\tau}^{\text{APBM}} \neq \boldsymbol{\theta}_t^{\text{APBM}}$), which requires adaptive estimation strategies [53].

The general APBM structure (40) can be simplified into an additive form with an explicitly controlled contribution of the data-based component (see "Data-Augmented Physics-Based Model With Controlled Additive Structure"). Bounding this contribution is an essential feature of the APBM, which prevents the data-based component from overruling the PBM component contribution and, thus, preserving APBM explainability. The APBM state dynamics model (40) together with the observation model [any of (1b), (2b), or (3b)] can be directly used for joint state $\boldsymbol{x}_t$ and parameter $\boldsymbol{\theta}_t^{\text{APBM}}$ estimation by a regular state estimator, such as the EKF or the UKF [5].

Training APBMs

Training APBMs can be performed under different paradigms depending on data availability, architecture, and assumptions regarding the stationarity of the system. Although the learning strategy can be supervised when $\mathbb{D}_s$ is available, we focus on the more common problem in the Bayesian filtering literature, where only noisy observations, $\mathbb{D}_u$, are accessible, setting up an unsupervised learning scenario. In this context, parameter estimation can be achieved by 1) obtaining and maximizing the marginal posterior $p(\boldsymbol{\theta} \mid \boldsymbol{y}_{1:T})$, often using the energy function $[\varphi_T(\boldsymbol{\theta}) = -\log p(\boldsymbol{y}_{1:T} \mid \boldsymbol{\theta}) - \log p(\boldsymbol{\theta})]$ [5]; 2) obtaining the joint posterior $p(\boldsymbol{x}_t, \boldsymbol{\theta} \mid \boldsymbol{y}_{1:T})$ [53]; or 3) obtaining a point estimate $\hat{\boldsymbol{\theta}}$ through deterministic optimization strategies, often aiming at maximizing the variational lower bound of the log-likelihood $p(\boldsymbol{y}_{1:T}; \boldsymbol{\theta})$ [8].

In [15] and [53], the authors opted for a state augmentation approach aiming at obtaining the joint posterior distribution $p(\boldsymbol{x}_t, \boldsymbol{\theta}_t \mid \boldsymbol{y}_{1:T})$ through Bayesian filtering recursion. For such, system states are augmented with the APBM parameters, as follows:

$$\begin{bmatrix} \boldsymbol{x}_{t+1} \\ \boldsymbol{\theta}_{t+1}^{\text{APBM}} \end{bmatrix} = \begin{bmatrix} g\big(f_t(\boldsymbol{x}_t),\, \boldsymbol{x}_t,\, \boldsymbol{d}_t; \boldsymbol{\theta}_t^{\text{APBM}}\big) \\ \boldsymbol{\theta}_t^{\text{APBM}} \end{bmatrix} + \begin{bmatrix} \boldsymbol{v}_t^{x} \\ \boldsymbol{v}_t^{\theta} \end{bmatrix} \tag{41}$$

where a near-constant state transition process is introduced for θ^{APBM}, allowing one to cast the APBM learning as a filtering problem. Here, $\boldsymbol{v}_t^{\theta}$ is a "small" noise and is introduced to avoid numerical issues [5]. Note that such noise can also allow $\boldsymbol{\theta}_t^{\text{APBM}}$ to drift over time and eventually evolve if the model is time varying.

Another important component regarding the APBM learning process is the need for a control mechanism to prevent the DNN component from overpowering the PBM. The prevention can be achieved by imposing appropriate constraints over $\boldsymbol{\theta}^{\text{APBM}}$. A Bayesian filtering-based approach is to introduce pseudo-observation equations into the observation model. Let $\bar{\boldsymbol{\theta}}$ be a vector such that $g\big(f_t(\boldsymbol{x}_t),\, \boldsymbol{x}_t,\, \boldsymbol{d}_t; \boldsymbol{\theta}_t^{\text{APBM}} = \bar{\boldsymbol{\theta}}\big) = f_t(\boldsymbol{x}_t)$; then, the observation model in (3b) can be augmented as

$$\begin{bmatrix} \boldsymbol{y}_t \\ \bar{\boldsymbol{\theta}} \end{bmatrix} = \begin{bmatrix} h_t(\boldsymbol{x}_t) \\ \boldsymbol{\theta}_t^{\text{APBM}} \end{bmatrix} + \begin{bmatrix} \boldsymbol{w}_t \\ \boldsymbol{w}_t^{\theta} \end{bmatrix} \tag{42}$$

where the bottom equation acts as a constraint, forcing $\boldsymbol{\theta}_t^{\text{APBM}}$ to be in the vicinity of $\bar{\boldsymbol{\theta}}$, and the distribution of $\boldsymbol{w}_t^{\theta}$ governs the tradeoff between regularization and data fit. In the case where the noise terms in the SS model defined in (41) and (42) are Gaussian, $\text{cov}\{\boldsymbol{\omega}_t^{\theta}\} = \boldsymbol{R}$, and $\text{cov}\{\boldsymbol{\omega}_t^{\theta}\} = \eta^{-1}\boldsymbol{I}$, the Bayesian filtering solution can be cast as a sequence of minimization problems for every time step t, as follows:

$$\begin{aligned} (\hat{\boldsymbol{x}}_t, \hat{\boldsymbol{\theta}}_t^{\text{APBM}}) = \underset{(\boldsymbol{x}_t, \boldsymbol{\theta}_t)}{\text{argmin}} & \big\| \boldsymbol{y}_t - h_t(\boldsymbol{x}_t) \big\|_{\boldsymbol{R}^{-1}}^2 + \eta \big\| \bar{\boldsymbol{\theta}} - \boldsymbol{\theta}_t \big\|^2 \\ & + \Big\| \boldsymbol{x}_t - g\big(f_t(\boldsymbol{x}_{t-1}),\, \boldsymbol{x}_{t-1},\, \boldsymbol{d}_{t-1}; \hat{\boldsymbol{\theta}}_{t-1}^{\text{APBM}}\big) \Big\|_{[\hat{\boldsymbol{P}}_{t|t-1}^{x}]^{-1}}^2 \\ & + \big\| \boldsymbol{\theta}_t - \hat{\boldsymbol{\theta}}_{t-1}^{\text{APBM}} \big\|_{[\hat{\boldsymbol{P}}_{t|t-1}^{\theta}]^{-1}}^2 \end{aligned} \tag{43}$$

Authorized licensed use limited to: The Chinese University of Hong Kong CUHK(Shenzhen). Downloaded on September 15,2025 at 16:58:26 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

where $\eta \geq 0$ controls the strictness of the regularization added by the pseudo-observation equation in (42).

Offline versus online training of APBMs

One interesting aspect of leveraging the Bayesian filtering approach for joint parameter and state estimation lies in its equivalence to a second-order Newton method [54], leading to fast convergence and making it suitable for both online and offline training. The training methodology seamlessly allows for either offline or online training or both (offline–online). In the offline scenario, the filtering with the augmented model can be performed over the multiple sequences (and multiple epochs) in $\mathbb{D}_u$ if system states, $\boldsymbol{x}_0$, are properly initialized for every data sequence in $\mathbb{D}_u$ [55]. Once training is completed, a standard Bayesian filtering approach can be used to update only the system states, $\boldsymbol{x}_t$, over time while keeping the APBM parameters fixed. In the online setting, the training approach becomes a conventional filtering problem that continuously adapts both states and model parameters over time, starting from some initial condition $\hat{\boldsymbol{x}}_0$, $\hat{\boldsymbol{\theta}}_0^{\text{APBM}}$. Again, the fast convergence of the Bayesian filter allows for quick reaction of the data-driven component over time, correcting the PBM to adapt to new conditions and improving the state estimation performance. This feature is extremely important when dealing with nonstationary dynamics that change over time. Both strategies can be combined, where the offline training solution is used as the initial condition for the online procedure, which, in turn, keeps updating both states and parameters during the test.

Discussion

The SS-oriented AI-augmented KF design relies on augmenting the physics-based component, namely the "deterministic" part of the state equation, with a data component. As seen in "Data-Augmented Physics-Based Model With Controlled Additive Structure," a solution might lead to an additive APBM state equation (S4). The data component is designed to extract a time-correlated component of the discrepancy between the pure PBM (3a) and the TM, which is usually embraced by the overbounded process noise $\boldsymbol{v}_t$.

Data-Augmented Physics-Based Model With Controlled Additive Structure

The general data-augmented physics-based model (APBM) in (40) can be particularized as an additive augmentation of the PBM [15], as follows:

$$\boldsymbol{x}_{t+1} = \boldsymbol{\phi}_0 f_t(\boldsymbol{x}_t) + \boldsymbol{\phi}_1 g(\boldsymbol{x}_t; \boldsymbol{\theta}^{\text{DNN}}) + \boldsymbol{v}_t^{\text{APBM}} \tag{S4}$$

where the APBM parameters $\boldsymbol{\theta}^{\text{APBM}} = [\boldsymbol{\phi}_0, \boldsymbol{\phi}_1, \boldsymbol{\theta}^{\text{DNN}}]$ may be estimated offline or online, with supervised or unsupervised training. To ensure that the data-based component does not overrule the PBM component, we can use constrained state estimation algorithms, where the deep neural network (DNN) parameters are encouraged to be close to a nominal value $\bar{\boldsymbol{\theta}}$ that keeps the DNN contribution limited. State estimation in this case is interpreted as a regularized optimization problem

$$(\hat{\boldsymbol{x}}_{1:t}, \boldsymbol{\theta}) = \underset{(\boldsymbol{x}_{1:t}, \boldsymbol{\theta})}{\operatorname{argmin}} \mathcal{L}_{\mathbb{D}_u}(\boldsymbol{y}_{1:t}, \bar{\boldsymbol{y}}_{1:t}) + \eta \mathcal{R}(\boldsymbol{x}_{1:t}, \boldsymbol{\theta})$$

where $\mathcal{L}_{\mathbb{D}_u}$ and $\mathcal{R}$ are cost and regularization functionals, $\eta \in \mathbb{R}_+$ is a regularization parameter governing the tradeoff between model fit and regularization, and $\bar{\boldsymbol{y}}_t \triangleq h_t(\boldsymbol{x}_t)$. In the context of Kalman filtering, the minimized cost function is the mean squared-error in recovering the state given the observations under the state evolution model in (S4). The DNN can be effectively controlled through the regularization term, for which several implementation options are possible [15]. The concept of the Kalman filter-based estimation of the APBM with controlled additive structure is described in Figure S4.

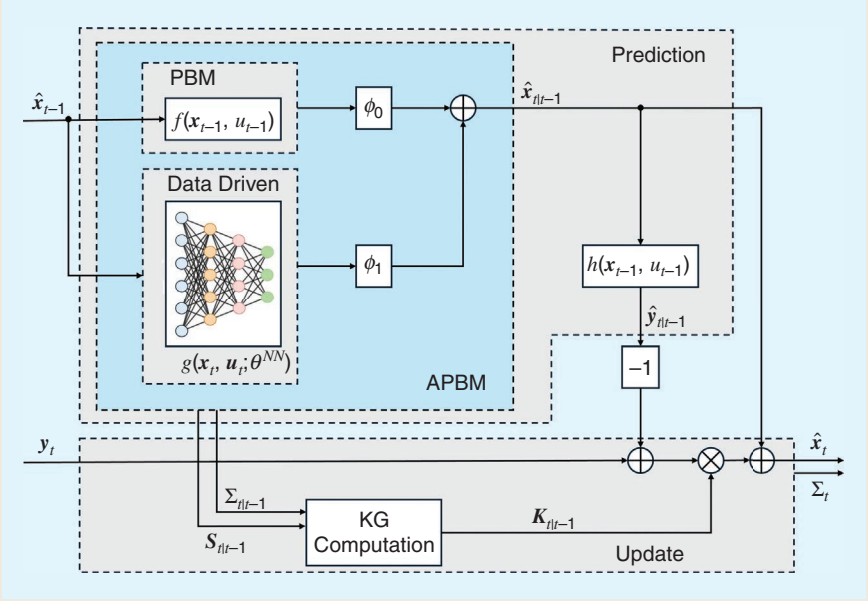

FIGURE S4. An APBM with additive structure.

Authorized licensed use limited to: The Chinese University of Hong Kong CUHK(Shenzhen). Downloaded on September 15,2025 at 16:58:26 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

As a consequence, the APBM process noise $\boldsymbol{v}_t^{\text{APBM}}$ has different statistical properties from the PBM $\boldsymbol{v}_t$, and for optimal performance of the KF, the noise properties have to be identified. In [56], the noise properties identification of the APBM was discussed and illustrated using correlation and maximum-likelihood methods. Utilizing the identified process noise covariance matrix in the KF led to significant improvement of the estimate consistency.

Following the terminology used in system identification [57], the PBM augmentation with a DNN belongs to block-oriented "slate-gray" models. The idea of block-oriented models is "to build up structures from simple building blocks" [57], which allows physical insight and a data-oriented flexible complement. Note that besides the DNN, the PBM can be augmented with the nonlinear autoregressive moving-average model, as in [58], where the data component is used for drift compensation. The APBM concept can also be used for deterministic models with a completely measured state, for characterizing unmodeled aspects of the dynamics.

Comparative study

The previous sections presented a set of design approaches and concrete algorithms fusing Kalman-type filtering and AI techniques. While all these methods tackle the same task of state estimation in dynamic systems, they notably vary in their strengths, requirements, and implications. To highlight the interplay among the approaches mentioned above and provide an understanding of how practitioners should prioritize one approach over the others, we next provide a comparative study. We divide this study into two parts. We first present a qualitative comparison, which pinpoints the conceptual differences among the approaches in light of the identified desired properties P1–P5 and challenges C1–C4. We then detail a quantitative study for a representative scenario of tracking in challenging dynamics to capture the regimes in which each approach is expected to be preferable.

Qualitative comparison

Here, we discuss the relationship and gains of the different approaches over one another in terms of figures of merit that are not quantifiable in the same sense that estimation performance on a given test bed is. Based on the identified desired properties P1–P5 and challenges C1–C4 of conventional Kalman-type algorithms, we focus on qualitative comparison in terms of domain knowledge, interpretability, uncertainty extraction, adaptability, the target family of SS models, and the learning framework. The comparison detailed below is summarized in Table 2.

Domain knowledge

The presented methodologies substantially vary in the required knowledge of the SS model, corresponding to challenges C1 and C2. The extreme cases are those of fully model-based filters (such as the KF and its variants) that require full and accurate knowledge of the SS model and those of end-to-end DNNs that are model agnostic (yet do not incorporate characterization when provided).

Among the methodologies representing AI-augmented KFs, the application of an external DNN to classic Kalman-type tracking typically requires the same level of domain knowledge needed to apply its classic counterpart. When the DNN is applied in parallel as a learned correction term, then the complete SS model should be known, while applying a learned preprocessing module can compensate for unknown and intractable observation modeling. Settings in which one has full knowledge of the observations model but does not know the state evolution model are the focus of methods based on learned state estimation and the SS-oriented DDM and PINN. Knowledge (though possibly approximated, as in C1) of the functions $f(\cdot)$ and $h(\cdot)$ is required by techniques that learn the KG, while some modeling of these functions is needed by SS-oriented approaches based on parameter learning and the APBM.

Interpretability

By *interpretability* we refer to the ability to explain the operation of each computation and associate its internal features with concrete meaning, as in P3. This high level of interpretability is naturally provided by purely model-based KF-type algorithms as well as by SS-oriented AI techniques that learn the parameters of a predetermined parametric SS model [51], similar to conventional system identification. On the contrary, algorithms employing end-to-end DNNs for state estimation are essentially black-box methods.

The level of interpretability varies when considering algorithms that augment classic algorithms with DNNs such that some of the computations are based on principled statistical models, while some are based on black-box data-driven pipelines. For instance, the internal features of task-oriented designs with integrated DNNs are exactly those of standard Kalman-type algorithms, while some of the internal computations—such as the computation of the KG in KalmanNet or the propagation of the prior state moments in DANSE—are carried out by black-box DNNs. A higher level of interpretability is provided when the DNN is applied in parallel to a fully model-based and interpretable algorithm for learned correction, as in [19].

Uncertainty

The ability to provide faithful uncertainty measures (as in P4) arises in Kalman-type algorithms that track of the error (posterior) covariance matrix. As the posterior update is an integral aspect of the KF update step, any algorithm that fully implements this computation provides uncertainty measures along with its state estimate. This property is exhibited by fully model-based Kalman-type filters as well as by some AI-aided KFs. Specifically, both SS-oriented designs, which implement conventional filtering on top of a learned state evolution model, and task-oriented architecture that augments the prior state prediction or utilizes DNNs external to model-based filters provide uncertainty in the update computation.

Authorized licensed use limited to: The Chinese University of Hong Kong CUHK(Shenzhen). Downloaded on September 15,2025 at 16:58:26 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

Among AI-aided algorithms that do not preserve the posterior covariance propagation of KFs, the ability to provide uncertainty varies. Generic end-to-end DNNs that track only the state do not offer such measures, while KF-inspired architectures are designed to output error covariances via their update DNNs. AI-aided KFs with learned KG (such as KalmanNet) are specifically designed to bypass the need to propagate second-order moments, such as the posterior covariance. Still, as noted in "Uncertainty Extraction From Learned Kalman Gain," in some settings, one can still extract uncertainty measures from their internal KG features.

Adaptability

As noted in P2, KF-type algorithms are adaptable to temporal variations in the statistical model. Assuming that one can identify and characterize the variations, the updated SS model parameters are simply substituted into the filter equations. For end-to-end DNNs, in which there is no explicit dependence on the SS model parameters, and these are implicitly embedded in the learned weights, adapting the SS models that deviate from those observed in training necessitates retraining; i.e., they are not adaptive.

Task-oriented AI-aided KFs are trained in a manner that is entangled with the operation of the augmented algorithm. While algorithms with integrated and external DNNs typically use the SS model parameters in their processing, their DNN modules are fixed and are suitable for the SS models observed in training. Accordingly, adaptations require retraining, where some level of variations can still be coped with using hypernetworks [41]. An exception is the usage of external DNNs for

Table 2. A qualitative comparison of the considered approaches.

Approach		SS Knowledge	Interpretability	Uncertainty	Adaptability	SS Model	Learning
End-to-end DNN	Generic	Fully model agnostic	Black box	Only outputs state	Not adaptive	Can be nonlinear and non-Gaussian	Supervised learning from large datasets
	KF-inspired (e.g., RKN [7])	Fully model agnostic	Connection of black-box DNNs	Estimates both state and uncertainty	Not adaptive	Can be nonlinear and non-Gaussian	Supervised learning from large datasets
External DNN	Learned preprocess	Requires state evolution	Input transformation is black box, while tracking based on the latent representation is fully interpretable	Tracks both state and uncertainty	Adapts to known variations only in the state evolution	State evolution should be simple (preferably linear) and Gaussian	Preferably supervised for end-to-end learning, though DNN can also be trained as a feature extractor, possibly unsupervised
	Learned correction (e.g., [19])	Requires (estimated) SS representation	Highly interpretable	Tracks both state and uncertainty	Designed for a specific SS model	Should be Gaussian with simple nonlinearities	Supervised learning from moderate datasets
Integrated DNN	Learned KG (e.g., KalmanNet [16])	Requires (estimates of) $h(\cdot)$ and $f(\cdot)$	Processing follows the same pipeline as standard EKF, with KG computation being black box	Can be extracted in some scenarios [40]	Requires retraining [21] or hypernetworks [41] for adaptivity	Can be nonlinear and non-Gaussian	Preferably supervised for end-to-end learning, can also be trained unsupervised with additional domain knowledge [21]
	Learned state estimation (e.g., DANSE [20])	Requires SS observation model and does not require SS state evolution model	Posterior update is interpretable and resembles that of SS-oriented KF, while state prediction is black box	Tracks both state and uncertainty	Requires retraining for adaptivity	State evolution model can be nonlinear, observation model should be linear, Gaussian	Unsupervised learning
SS-oriented DNN	DDM (e.g., fully DDM or PINN [29])	Fully model agnostic (DNN) or some knowledge of the model required (PINN) Observation model is assumed to be known	State evolution is black box, while the filter operation is interpretable	Tracks both state and uncertainty	Not adaptive	Can be nonlinear and non-Gaussian	Supervised learning
	Parameter learning (e.g., [51])	Requires models of $h(\cdot)$ and $f(\cdot)$	Fully interpretable	Tracks both state and uncertainty	Not adaptive	Can be nonlinear and non-Gaussian	Unsupervised learning
	APBM (e.g., [15])	Requires models of $h(\cdot)$ and $f(\cdot)$	Highly interpretable	Tracks both state and uncertainty	Fully adaptive	Can be nonlinear and non-Gaussian	Unsupervised learning
Model based	Kalman-type filters (e.g., EKF)	Requires (accurate) SS representation	Fully interpretable	Tracks both state and uncertainty	Fully adaptive	Should be Gaussian with mild nonlinearities	Fully model based

Authorized licensed use limited to: The Chinese University of Hong Kong CUHK(Shenzhen). Downloaded on September 15,2025 at 16:58:26 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

Table 3. The MSE, in decibels, of a Lorenz attractor with sampling mismatch.

Noise	EKF	PF	KalmanNet	DANSE	APBM (Offline)	APBM (Online)
–0.024 ± 0.049	–6.316 ± 0.135	–5.333 ± 0.136	–11.106 ± 0.224	–10.115 ± 0.163	–9.457 ± 0.231	–7.452 ± 0.548

feature extraction, which is typically designed to map complex observations into a simplified observation model and thus is not affected by variations in the state evolution model. Among SS-oriented approaches, adaptivity is provided by the design of the APBM, while alternatives based on, e.g., DDM or PINN architectures need retraining when the state evolution statistics change.

SS model type

A key distinct property among the presented methodologies lies in the family of SS models for which each algorithm is suitable. As noted in C3, fully model-based KF-type algorithms are most suitable for Gaussian SS models with simple and, preferably, not highly nonlinear state evolution and observation models. On the opposite edge of the spectrum are end-to-end model-agnostic DNNs, which can be applied in complex and intractable dynamic systems, provided with sufficient data corresponding to that task.

Among task-oriented AI-augmented KFs, architectures employing external DNNs as learned correction terms are suitable for similar SS models as classic KFs, being geared toward settings where the latter is applicable. External DNNs that preprocess observations facilitate coping with complex observation models, while integrated DNNs for learned state estimation overcome complex state evolution models, with each approach requiring the remainder of the SS model to be simple (preferably linear), fully known, and Gaussian. Augmenting KFs with learned KG notably enhances the family of applicable SS models, not requiring Gaussianity and modeling of any of the noise terms and learning to cope with nonlinear and possibly approximated state evolution and observation functions. The same holds for the reviewed SS-oriented methods that can all cope with nonlinear and non-Gaussian dynamics.

Learning framework

The learning framework is specific to algorithms that incorporate AI and encapsulates the amount of data and the reliance on its labeling. Accordingly, fully model-based algorithms that rely on given mathematical modeling of the SS representation do not involve learning in their formulation, while black-box DNNs typically require learning from large volumes of labeled datasets.

Designs combining partial domain knowledge with AI typically result in a dominant inductive bias that allows training with limited data, compared to black-box DNNs. Methods based on SS-oriented DDMs/PINNs, external DNNs, and learned integrated KG DNNs typically require labeled data, though for learned preprocessing and for KalmanNet, one can also train unsupervised in some settings. Methods such as DANSE, SS-oriented parameter learning, and APBMs are designed to be trained based solely on observations, i.e., unsupervised.

Quantitative comparison

To illustrate the performance of the considered state estimation algorithms, we provide a dedicated experimental study.[4] We consider the task of tracking the nonlinear movement of a free particle in 3D space ($m = 3$) from noisy position observations. The Lorenz attractor, a chaotic solution to the Lorenz system of ordinary differential equations, defines the continuous-time state evolution of the particle's trajectory. To enhance the challenge of this tracking case, we introduce additional uncertainty due to a sampling time mismatch in the observation process. While the underlying ground truth synthetic trajectory is generated using a high-resolution time interval ($\Delta\tau = 10^{-5}$), the tracking filter can access only noisy observations decimated at a rate of $1/2000$, resulting in a decimated process with $\Delta t = 0.02$.

To ensure a fair evaluation, we restrict any populated evolution model of the tracking filter, if it exists, to discrete time, with $\Delta t \geq 0.02$. Further details about the Lorenz attractor evolution model can be found in [35]. The averaged MSE values and their standard deviation values for filtering 10 sequences with a length of $T = 3{,}000$ time steps are reported in Table 3. There, we compare the MSE achieved by using the noisy observations as the state estimate (noise), filtering via the model-based EKF and PF, the DNN-integrated KalmanNet and DANSE, and two forms of the SS-oriented APBM, employing offline and online learning. In the comparative performance reported in Table 3, the model-based filters, i.e., the EKF and PF, manage to improve upon using noisy observations but suffer from an error floor due to their sampling mismatch. All considered AI-aided filters manage to improve upon this error floor. Specifically, KalmanNet, which is trained in a supervised manner, achieves the best performance, improving by approximately 5 dB in the MSE compared to the model-based algorithms. Among the AI-aided filters that are trained from unlabeled data, DANSE achieves the best performance. The APBM, which is designed to boost adaptivity, achieves an MSE within a small gap of DANSE when trained offline.

The above quantitative results are evaluated in a controlled experimental setup, which allows capitalizing on the differences in performance arising from the inherent properties of the considered algorithms. In that sense, they complement the

[4]The source code for all the algorithms and the hyperparameters can be found online at https://github.com/ShlezingerLab/AI_Aided_KFs.

Authorized licensed use limited to: The Chinese University of Hong Kong CUHK(Shenzhen). Downloaded on September 15,2025 at 16:58:26 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

conceptual comparison provided in Table 2, in revealing the interplay among the reviewed methods for combining deep learning with classic KF-type filtering. We note that additional evaluations of these methodologies in application-specific settings and with real-world data are reported in the literature. Representative examples include the comparison of KFs, RNNs, and augmented KFs for brain–machine interfaces in [22]; evaluation of DNN-aided KFs for coastline monitoring in [23]; and tracking results comparing DNN-aided KFs with limited training data in [37].

Future research directions

The tutorial-style presentation of designs, algorithms, and experiments of AI-aided KFs indicates potential gains of proper fusion of model-based tracking algorithms and data-driven deep learning techniques. These, in turn, give rise to several core research directions that can be explored to further unveil the potential, strengths, and prospective use cases of such designs. Exploring these directions is expected to further advance the development and understanding of algorithmic tools to jointly leverage classic SS model-based KF-type algorithms with emerging deep learning techniques and preserve the individual strengths of each approach that has a bearing on the potential of alleviating some of the core challenges shared by a broad range of technologies tracking dynamic systems and state estimation. We thus conclude this article with a discussion of some of the open challenges that can serve as future research directions.

1) *Time-varying SS models and adaptation*: A practical state estimation method might need to cater to a scenario where the observation model (1b) is mismatched between training and testing stages and/or the observation or state evolution functions are time varying without a clear pattern. Typically, AI-aided KFs are tied to a fixed observation model, typically not time varying and that remains the same between the training stage and inference stage. Therefore, future research may aim to design AI-aided state estimation methods that adapt to time-varying SS models, mainly for time-varying observation models.
2) *Non-Markovian SS models*: Throughout this article and being consistent with usual practice, the state evolution model is Markovian [see (1a)]. Naturally, it is a quest to design methods that can exploit short- and long-term memories in state evolution, which means non-Markovian state evolution.
3) *Non-Gaussian SS models*: Most of the directions discussed focus on Gaussian noises in SS models. Extensions to non-Gaussian noises will require more complex algorithms with high computational complexity, which is challenging, especially when learning the large-scale DNN parameters.
4) *Distributed AI-aided KFs*: While there exists considerable research on distributed KFs, such as the work of [32] for sensor networks, there is currently little attention to designing distributed AI-aided KFs. Designing them for federated learning and edge computing can be a new research direction.
5) *Robust training*: Outliers appearing in the SS models, mainly in the observation model (1b), may substantially affect the training process. The approaches should address this by providing robust training.

Acknowledgment

This work has been partially supported by the European Research Council (ERC), under ERC Starting Grant 101163973 (FLAIR); the National Science Foundation, under Awards ECCS-1845833 and CCF-2326559; and the Czech Republic Ministry of Education, Youth, and Sports through the Robotics and Advanced Industrial Production project, Grant CZ.02.01.01/00/22_008/0004590.

Authors

Nir Shlezinger (nirshl@bgu.ac.il) received his Ph.D. degree in 2017 from Ben-Gurion University, 8410501 Beersheba, Israel, where he is an assistant professor in the School of Electrical and Computer Engineering. He was awarded the Feinberg Graduate School prize for outstanding postdoctoral research achievements, the 2024 Krill award for young researchers, the Toronto Early Career Award, and the 2024 IEEE Communications Society Fred W. Ellersick prize. His research interests include signal processing, machine learning, and communications. He is a Senior Member of IEEE.

Guy Revach (grevach@ethz.ch) received his M.Sc. degree in 2017 from the Andrew and Erna Viterbi Department of Electrical and Computer Engineering, Technion–Israel Institute of Technology, Haifa, Israel. He is a Ph.D. candidate at the Institute for Signal and Information Processing, ETH Zürich, 8092 Zürich, Switzerland, supervised by Dr. Hans-Andrea Loeliger. His research interests include the intersection of machine learning with signal processing, explicitly combining sound theoretical principles from classical signal processing with state-of-the-art machine learning algorithms for tracking and detection problems. He is a Senior Member of IEEE.

Anubhab Ghosh (anubhabg@kth.se) received his M.Sc. degree in information and network engineering in 2020 from KTH Royal Institute of Technology, SE-100 44 Stockholm, Sweden, where he is a Ph.D. student in the Division of Information Science and Engineering. His research interests include the intersection of machine learning and dynamical systems, including generative models, sequential learning, system identification, filtering, and estimation using machine learning. He is a Graduate Student Member of IEEE.

Saikat Chatterjee (sach@kth.se) received his Ph.D. degree in 2009 from the Indian Institute of Science, Bengaluru 560012, India. He is an associate professor at the School of Electrical Engineering and Computer Science, KTH Royal Institute of Technology, SE-100 44 Stockholm, Sweden. He is on the editorial board of *Digital Signal Processing* (Elsevier) and *EURASIP Journal on Advances in Signal Processing*. He was the chair of the European Association for Signal Processing Special Area Team on Signal and Data Analytics for Machine Learning. He and a coauthor won the Best

Authorized licensed use limited to: The Chinese University of Hong Kong CUHK(Shenzhen). Downloaded on September 15,2025 at 16:58:26 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

Student Paper Award at ICASSP 2010. His research interests include statistical signal processing, machine learning, medical data analysis, data analytics, and speech, audio, and image processing. He is a Senior Member of IEEE.

Shuo Tang (tang.shu@northeastern.edu) received his M.S. degree in mechanical engineering in 2020 from Northeastern University, Boston, MA 02115 USA, where he is a Ph.D. candidate in electrical and computer engineering. His research interests include global navigation satellite system signal processing, sensor fusion, and computational statistics. He is a Student Member of IEEE.

Tales Imbiriba (talesim@gmail.com) received his Ph.D. degree from the Department of Electrical Engineering, Federal University of Santa Catarina, Florianopolis, Brazil, in 2016. He is an assistant professor in the Department of Computer Science, University of Massachusetts Boston, Boston, MA 01003 USA. His research interests include Bayesian inference, online learning, and physics-guided machine learning, with applications in human-centered technologies, remote sensing of the environment, and enhanced security technologies. He is a Member of IEEE.

Jindrich Dunik (dunikj@kky.zcu.cz) received his Ph.D. degree in 2008 from the University of West Bohemia, 30100 Plzeň, Czech Republic. He is an associate professor in the Department of Cybernetics, University of West Bohemia and a senior scientist with Honeywell International Aerospace Advanced Technology Europe. He is a member of the IEEE Aerospace and Electronic Systems Society (AESS) Navigation System Panel and serves as an associate editor of *IEEE Transactions on Aerospace and Electronic Systems*. He is a coauthor of more than 80 technical papers (both journal and conference) and patents. He received the Honeywell Technology Achievement Award, the Werner von Siemens Excellence Award, and the AESS Harry Rowe Mimno Award. His research interests include state estimation, system identification, inertial and satellite-based navigation systems, and integrity monitoring methods.

Ondrej Straka (straka30@kky.zcu.cz) received his Ph.D. degree in 2004 from the University of West Bohemia, 30100 Plzeň, Czech Republic. He is an associate professor in the Department of Cybernetics, University of West Bohemia, where he is the head of the Identification and Decision-Making Research Group, NTIS Research Center. He serves as an associate editor of *IEEE Transactions on Aerospace and Electronic Systems* and has coauthored over 140 journal and conference papers. He received the Werner von Siemens Excellence Award in 2014 for important results in basic research. His research interests include local and global nonlinear state estimation methods, system identification, performance evaluation, and fault detection. He is a Member of IEEE.

Pau Closas (closas@ece.neu.edu) received his Ph.D. degree in electrical engineering in 2009 from the University of Barcelona, Barcelona, Spain. He is an associate professor of electrical and computer engineering at Northeastern University, Boston, MA 02115 USA. He is the editor-in-chief of *IEEE Aerospace and Electronic Systems Magazine*. He is the recipient of several awards, including the 2014 European Association for Signal Processing Best Ph.D. Thesis Award, 2016 Institute of Navigation Early Achievement Award, 2019 National Science Foundation CAREER Award, and 2022 IEEE Aerospace and Electronic Systems Society Harry Rowe Mimno Award. His research interests include statistical signal processing and machine learning, with applications to positioning and localization systems. He is a Senior Member of IEEE.

Yonina C. Eldar (yonina.eldar@weizmann.ac.il) received her Ph.D. degree in electrical engineering and computer science from the Massachusetts Institute of Technology in 2002. She is a professor in the Department of Mathematics and Computer Science, Weizmann Institute of Science, 7610001 Rehovot, Israel, where she heads the Center for Biomedical Engineering and Signal Processing and holds the Dorothy and Patrick Gorman Professorial Chair. She is the editor-in-chief of *Foundations and Trends in Signal Processing* and a member of several IEEE technical and award committees, and she heads the Committee for Promoting Gender Fairness in Higher Education Institutions in Israel. She is a fellow of the European Association for Signal Processing, Asia-Pacific Artificial Intelligence Association, and 8400 Health Network. She received the IEEE Signal Processing Society Technical Achievement Award (2013), IEEE Aerospace and Electronic Systems Society Fred Nathanson Memorial Radar Award (2014), and IEEE Kiyo Tomiyasu Award (2016). She is a Fellow of IEEE.

References

[1] R. E. Kalman, "A new approach to linear filtering and prediction problems," *J. Basic Eng.*, vol. 82, no. 1, pp. 35–45, 1960, doi: 10.1115/1.3662552.

[2] S. F. Schmidt, "The Kalman filter-its recognition and development for aerospace applications," *J. Guid. Control*, vol. 4, no. 1, pp. 4–7, 1981, doi: 10.2514/3.19713.

[3] M. S. Grewal and A. P. Andrews, "Applications of Kalman filtering in aerospace 1960 to the present [Historical Perspectives]," *IEEE Control Syst. Mag.*, vol. 30, no. 3, pp. 69–78, Jun. 2010, doi: 10.1109/MCS.2010.936465.

[4] J. Durbin and S. J. Koopman, *Time Series Analysis by State Space Methods*, vol. 38. London, U.K.: Oxford Univ. Press, 2012.

[5] S. Särkkä and L. Svensson, *Bayesian Filtering and Smoothing*, vol. 17. Cambridge, U.K.: Cambridge Univ. Press, 2023.

[6] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning.* Cambridge, MA, USA: MIT Press, 2016.

[7] P. Becker, H. Pandya, G. Gebhardt, C. Zhao, C. J. Taylor, and G. Neumann, "Recurrent Kalman networks: Factorized inference in high-dimensional deep feature spaces," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 544–552.

[8] R. Krishnan, U. Shalit, and D. Sontag, "Structured inference networks for nonlinear state space models," in *Proc. 31st AAAI Conf. Artif. Intell.*, 2017, pp. 2101–2109, doi: 10.1609/aaai.v31i1.10779.

[9] S. Kuutti, R. Bowden, Y. Jin, P. Barber, and S. Fallah, "A survey of deep learning applications to autonomous vehicle control," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 2, pp. 712–733, Feb. 2021, doi: 10.1109/TITS.2019.2962338.

[10] J. Chen and X. Ran, "Deep learning with edge computing: A review," *Proc. IEEE*, vol. 107, no. 8, pp. 1655–1674, Aug. 2019, doi: 10.1109/JPROC.2019.2921977.

[11] N. Shlezinger, J. Whang, Y. C. Eldar, and A. G. Dimakis, "Model-based deep learning," *Proc. IEEE*, vol. 111, no. 5, pp. 465–499, May 2023, doi: 10.1109/JPROC.2023.3247480.

[12] A. Klushyn, R. Kurle, M. Soelch, B. Cseke, and P. van der Smagt, "Latent matters: Learning deep state-space models," in *Proc. 35th Int. Conf. Neural Inf. Process. Syst.*, 2021, pp. 10,234, 10,245.

[13] H. Coskun, F. Achilles, R. DiPietro, N. Navab, and F. Tombari, "Long short-term memory Kalman filters: Recurrent neural estimators for pose regularization," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2017, pp. 5524–5532.

[14] D. Gedon, N. Wahlström, T. B. Schön, and L. Ljung, "Deep state space models for nonlinear system identification," *IFAC-PapersOnLine*, vol. 54, no. 7, pp. 481–486, 2021, doi: 10.1016/j.ifacol.2021.08.406.

Authorized licensed use limited to: The Chinese University of Hong Kong CUHK(Shenzhen). Downloaded on September 15,2025 at 16:58:26 UTC from IEEE Xplore. Restrictions apply.

This article has been accepted for inclusion in a future issue of this journal. Content is final as presented, with the exception of pagination.

[15] T. Imbiriba, O. Straka, J. Duník, and P. Closas, "Augmented physics-based machine learning for navigation and tracking," *IEEE Trans. Aerosp. Electron. Syst.*, vol. 60, no. 3, pp. 2692–2704, Jun. 2024, doi: 10.1109/TAES.2023.3328853.

[16] G. Revach, N. Shlezinger, X. Ni, A. L. Escoriza, R. J. van Sloun, and Y. C. Eldar, "KalmanNet: Neural network aided Kalman filtering for partially known dynamics," *IEEE Trans. Signal Process.*, vol. 70, pp. 1532–1547, 2022, doi: 10.1109/TSP.2022.3158588.

[17] I. Buchnik, G. Revach, D. Steger, R. J. van Sloun, T. Routtenberg, and N. Shlezinger, "Latent-KalmanNet: Learned Kalman filtering for tracking from high-dimensional signals," *IEEE Trans. Signal Process.*, vol. 72, pp. 352–367, 2023, doi: 10.1109/TSP.2023.3344360.

[18] K. Pratik, R. A. Amjad, A. Behboodi, J. B. Soriaga, and M. Welling, "Neural augmentation of Kalman filter with hypernetwork for channel tracking," in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, 2021, pp. 1–6, doi: 10.1109/GLOBECOM46510.2021.9685798.

[19] V. G. Satorras, Z. Akata, and M. Welling, "Combining generative and discriminative models for hybrid inference," in *Proc. 33rd Int. Conf. Neural Inf. Process. Syst.*, 2019, pp. 13,802–13,812.

[20] A. Ghosh, A. Honoré, and S. Chatterjee, "DANSE: Data-driven non-linear state estimation of model-free process in unsupervised learning setup," *IEEE Trans. Signal Process.*, vol. 72, pp. 1824–1838, 2024, doi: 10.1109/TSP.2024.3383277.

[21] G. Revach, N. Shlezinger, T. Locher, X. Ni, R. J. G. van Sloun, and Y. C. Eldar, "Unsupervised learned Kalman filtering," in *Proc. Eur. Signal Process. Conf. (EUSIPCO)*, 2022, pp. 1571–1575.

[22] L. Cubillos et al., "Exploring the trade-off between deep-learning and explainable models for brain-machine interfaces," in *Proc. Adv. Neural Inf. Process. Syst.*, 2025, vol. 37, pp. 133,975–133,998.

[23] S. N. Aspragkathos, G. C. Karras, and K. J. Kyriakopoulos, "Event-triggered image moments predictive control for tracking evolving features using UAVs," *IEEE Robot. Autom. Lett.*, vol. 9, no. 2, pp. 1019–1026, Feb. 2024, doi: 10.1109/LRA.2023.3339064.

[24] A. Milstein, G. Revach, H. Deng, H. Morgenstern, and N. Shlezinger, "Neural augmented Kalman filtering with Bollinger bands for pairs trading," *IEEE Trans. Signal Process.*, vol. 72, pp. 1974–1988, 2024, doi: 10.1109/TSP.2024.3385984.

[25] A. Gu et al., "Combining recurrent, convolutional, and continuous-time models with linear state-space layers," in *Proc. 35th Int. Conf. Neural Inf. Process. Syst.*, 2021, pp. 572–585.

[26] N. Shlezinger and T. Routtenberg, "Discriminative and generative learning for linear estimation of random signals [Lecture Notes]," *IEEE Signal Process. Mag.*, vol. 40, no. 6, pp. 75–82, Sep. 2023, doi: 10.1109/MSP.2023.3271431.

[27] L. Zhou et al., "KFNet: Learning temporal camera relocalization using Kalman filtering," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 4919–4928, doi: 10.1109/CVPR42600.2020.00497.

[28] S. Jouaber, S. Bonnabel, S. Velasco-Forero, and M. Pilte, "NNAKF: A neural network adapted Kalman filter for target tracking," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, 2021, pp. 4075–4079, doi: 10.1109/ICASSP39728.2021.9414681.

[29] M. Raissia, P. Perdikarisb, and G. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," *J. Comput. Phys.*, vol. 378, pp. 686–707, Feb. 2019, doi: 10.1016/j.jcp.2018.10.045.

[30] A. Vaswani et al., "Attention is all you need," in *Proc. 31st Conf. Adv. Neural Inf. Process. Syst.*, 2017, pp. 5998–6008.

[31] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," 2023, *arXiv:2312.00752*.

[32] R. Olfati-Saber, "Distributed Kalman filtering for sensor networks," in *Proc. IEEE Conf. Decis. Control*, 2007, pp. 5492–5498, doi: 10.1109/CDC.2007.4434303.

[33] N. Shlezinger, Y. C. Eldar, and S. P. Boyd, "Model-based deep learning: On the intersection of deep learning and optimization," *IEEE Access*, vol. 10, pp. 115,384–115,398, 2022, doi: 10.1109/ACCESS.2022.3218802.

[34] L. Xu and R. Niu, "EKFNet: Learning system noise statistics from measurement data," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, 2021, pp. 4560–4564, doi: 10.1109/ICASSP39728.2021.9415083.

[35] G. Revach, X. Ni, N. Shlezinger, R. J. van Sloun, and Y. C. Eldar, "RTSNet: Learning to smooth in partially known state-space models," *IEEE Trans. Signal Process.*, vol. 71, pp. 4441–4456, 2023, doi: 10.1109/TSP.2023.3329964.

[36] G. Choi, J. Park, N. Shlezinger, Y. C. Eldar, and N. Lee, "Split-KalmanNet: A robust model-based deep learning approach for state estimation," *IEEE Trans. Veh. Technol.*, vol. 72, no. 9, pp. 12,326–12,331, Sep. 2023, doi: 10.1109/TVT.2023.3270353.

[37] S. Chen, Y. Zheng, D. Lin, P. Cai, Y. Xiao, and S. Wang, "MAML-KalmanNet: A neural network-assisted Kalman filter based on model-agnostic meta-learning," *IEEE Trans. Signal Process.*, vol. 73, pp. 988–1003, 2025, doi: 10.1109/TSP.2025.3540018.

[38] J. Wang, X. Geng, and J. Xu, "Nonlinear Kalman filtering based on self-attention mechanism and lattice trajectory piecewise linear approximation," 2024, *arXiv:2404.03915*.

[39] A. N. Putri, C. Machbub, D. Mahayana, and E. Hidayat, "Data driven linear quadratic Gaussian control design," *IEEE Access*, vol. 11, pp. 24,227–24,237, 2023, doi: 10.1109/ACCESS.2023.3254879.

[40] Y. Dahan, G. Revach, J. Dunik, and N. Shlezinger, "Uncertainty quantification in deep learning based Kalman filters," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, 2024, pp. 13,121–13,125, doi: 10.1109/ICASSP48485.2024.10447987.

[41] X. Ni, G. Revach, and N. Shlezinger, "Adaptive KalmanNet: Data-driven Kalman filter with fast adaptation," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, 2024, pp. 5970–5974, doi: 10.1109/ICASSP48485.2024.10447012.

[42] Z. Fan, D. Shen, Y. Bao, K. Pham, E. Blasch, and G. Chen, "RNN-UKF: Enhancing hyperparameter auto-tuning in unscented Kalman filters through recurrent neural networks," in *Proc. IEEE Int. Conf. Inf. Fusion*, 2024, pp. 1–8, doi: 10.23919/FUSION59988.2024.10706523.

[43] D. Masti and A. Bemporad, "Learning nonlinear state–space models using autoencoders," *Automatica*, vol. 129, Jul. 2021, Art. no. 109666, doi: 10.1016/j.automatica.2021.109666.

[44] A. Gorji and M. Menhaj, "Identification of nonlinear state space models using an MLP network trained by the EM algorithm," in *Proc. IEEE Int. Joint Conf. Neural Netw.*, 2008, pp. 53–60, doi: 10.1109/IJCNN.2008.4633766.

[45] J. Suykens, B. Moor, and J. Vandewalle, "Nonlinear system identification using neural state space models, applicable to robust control design," *Int. J. Control*, vol. 62, no. 1, pp. 129–152, 1995, doi: 10.1080/00207179508921536.

[46] M. Fraccaro, S. Sønderby, U. Paquet, and O. Winther, "Sequential neural models with stochastic layers," in *Proc. 30th Int. Conf. Neural Inf. Process. Syst. (NIPS)*, 2016, pp. 2207–2215.

[47] F. Arnold and R. King, "State–space modeling for control based on physics-informed neural networks," *Eng. Appl. Artif. Intell.*, vol. 101, May 2021, Art. no. 104195, doi: 10.1016/j.engappai.2021.104195.

[48] M. Schoukens, "Improved initialization of state-space artificial neural networks," in *Proc. Eur. Control Conf. (ECC)*, 2021, pp. 1913–1918, doi: 10.23919/ECC54610.2021.9655207.

[49] M. Forgione and D. Piga, "Model structures and fitting criteria for system identification with neural networks," in *Proc. IEEE Int. Conf. Appl. Inf. Commun. Technol.*, 2020, pp. 1–6, doi: 10.1109/AICT50176.2020.9368834.

[50] L. Ljung, C. Andersson, K. Tiels, and T. B. Schön, "Deep learning and system identification," *IFAC-PapersOnLine*, vol. 53, no. 2, pp. 1175–1181, 2020, doi: 10.1016/j.ifacol.2020.12.1329.

[51] Y. Bao, J. M. Velni, A. Basina, and M. Shahbakhti, "Identification of state-space linear parameter-varying models using artificial neural networks," *IFAC-PapersOnLine*, vol. 53, no. 2, pp. 5286–5291, 2020, doi: 10.1016/j.ifacol.2020.12.1209.

[52] S. Tang, T. Imbiriba, J. Duník, O. Straka, and P. Closas, "Augmented physics-based models for high-order Markov filtering," *Sensors*, vol. 24, no. 18, 2024, Art. no. 6132, doi: 10.3390/s24186132.

[53] T. Imbiriba, A. Demirkaya, J. Duník, O. Straka, D. Erdoğmuş, and P. Closas, "Hybrid neural network augmented physics-based models for nonlinear filtering," in *Proc. 25th Int. Conf. Inf. Fusion*, 2022, pp. 1–6, doi: 10.23919/FUSION49751.2022.9841291.

[54] J. Humpherys, P. Redd, and J. West, "A fresh look at the Kalman filter," *SIAM Rev.*, vol. 54, no. 4, pp. 801–823, 2012, doi: 10.1137/100799666.

[55] A. Bemporad, "Recurrent neural network training with convex loss and regularization functions by extended Kalman filtering," *IEEE Trans. Autom. Control*, vol. 68, no. 9, pp. 5661–5668, Sep. 2023, doi: 10.1109/TAC.2022.3222750.

[56] J. Dunik, O. Straka, O. Kost, S. Tang, T. Imbiriba, and P. Closas, "Noise identification for data-augmented physics-based state-space models," in *Proc. IEEE Workshop Signal Process. Syst. (SiPS)*, 2024, pp. 101–106, doi: 10.1109/SiPS62058.2024.00026.

[57] L. Ljung, "Perspectives on system identification," *IFAC Proc. Volumes*, vol. 41, no. 2, pp. 7172–7184, 2008, doi: 10.3182/20080706-5-KR-1001.01215.

[58] D. Li, J. Zhou, and Y. Liu, "Recurrent-neural-network-based unscented Kalman filter for estimating and compensating the random drift of MEMS gyroscopes in real time," *Mech. Syst. Sig. Process.*, vol. 147, Jan. 2021, Art. no. 107057, doi: 10.1016/j.ymssp.2020.107057.

SP

Authorized licensed use limited to: The Chinese University of Hong Kong CUHK(Shenzhen). Downloaded on September 15,2025 at 16:58:26 UTC from IEEE Xplore. Restrictions apply.